

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

1. SQL je jezik koji traži podatke od baze podataka. To nije proceduralni jezik jer opisuje podatke koje treba prikazati, ukloniti ili unijeti, a ne kako da se izvede ta operacija. Za relacioni model podataka je postavljeno 13 Codd-ovih pravila(0-12).
2. Naj popularnije SQL implementacije su: MS Access(DBMS za PC,lak je za upotrebu, možemo koristiti GUI alate ili ručno unijeti SQL iskaze), Personal Oracle 7(koji predstavlja bazu podataka većih dimenzija, koristimo da predstavimo SQL sa komandnim linijama i tehnike manipulisanja bazama podataka. U SQL-u sa komandnim linijama jednostavni samostalni iskazi SQL-a se unose u Oracle-ov SQL* Plus alat. Ovaj alat vraća podatke na ekran da bi korisnik mogao da ih vidi, ili izvršava odgovarajuću akciju nad bazom podataka.
3. ODBC (Open DataBase Conectivity) je tehnologija koja omogućava Windows programima da pristupe bazama podataka.
4. Upitni blok se formira korištenjem klauzula: SELECT, FROM i WHERE
Klauzule SELECT i FROM omogućavaju upitu da prikaže tražene podatke. U WHERE klauzuli se navode uslovi. Uslovi nam omogućavaju da formiramo upite selekcije.
5. Operatori se dijele u šest grupa: aritmetičke, operatore poređenja, operatore za karaktere (stringovne), logičke, skupovne i mješovite. Aritmetički operatori su operator sabiranja, oduzimanja, množenja, dijeljenja i moduo(kao rezultat daje ostatak pri dijeljenju).Operatori poređenja su:znak jednakosti(=), operatori veće,veće i jednako,manje, manje i jednako(>,>=,<,<=) i operatori poređenja po nejednakosti(<> ili !=). U operatore za stringove spadaju operator LIKE, podvlaka, operatori spajanja. Logički operatori su : AND,OR, i NOT. U skupovne operatore spadaju operatori: UNION, UNION ALL, INTERSECT i MINUS dok su IN i BETWEEN mješoviti operatori.
6. Operator I N predstavlja skraćenicu za operator OR dok je BETWEEN skraćenica za operator AND.
7. Agregatne funkcije nazivaju se još i funkcijama grupe. Kao rezultat daju vrijednost baziranu na vrijednostima iz kolone.Agregatne funkcije se ne mogu navoditi u WHERE klauzuli. U grupu agregatnih funkcija spadaju sljedeće funkcije: COUNT, SUM, MIN, MAX,AVG, VARIANCE, STDEV.
8. Sve važnije implementacije SQL-a uključuju funkcije za vrijeme i datum. To su sljedeće funkcije: ADD_MONTHS(dodaje broj mjeseci izabranom datumu), LAST_DAY (kao rezultat daje posljednji dan u mjesecu), MONTHS_BETWEEN (ukoliko je potrebno da saznamo koliko je mjeseci između mjeseca x i mjeseca y, NEW_TIME (kada postoji potreba za usklađivanjem vremena prema vremenskoj zoni u kojoj se nalazimo), NEXT_DAY (ovom funkcijom dobijamo prvi dan sedmice) i SYS_DATE (kojom dobijamo sistemski datum i vrijeme)
9. Aritmetičke funkcije su: ABS (rezultat koji vraća ova funkcija je apsolutna vrijednost broja), CEIL (vraća najmanju cjelobrojnu vrijednost koja je veća ili jednaka argumentu), FLOOR (vraća kao rezultat najveću cjelobrojnu vrijednost koja je jednaka ili manja od argumenta),COS,SIN,TAN, EXP (stepenovanje),LN,LOG, MOD, POWER, SIGN,SQRT (rezultat koji daje funkcija je kvadratni korijen argumenta).
10. To su funkcije: CONCAT (spaja dva stringa), INITCAP(postavlja veliko slovo na početak riječi dok su sva ostala slova mala), LOWER (ispisuje cijelu riječ u malim

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

- slovima), UPPER (ova funkcija radi suprotno), SUBSTR- funkcija koja omogućava da izdvojimo dio pretraživanog stringa. Prvi argument u funkciji je string koji se pretražuje, drugi argument je pozicija prvog karaktera koji treba prikazati i treći argument je broj karaktera koje treba prikazati.
11. Klauzula STARTING WITH je dodatak WHERE klauzuli i ponaša se isto kao i operator LIKE (<izraz>%>). Znači klauzulu STARTING WITH koristimo da ukoliko želimo da pronađemo sve podatke iz baze podataka koji odgovaraju našem uzorku.
12. Klauzula ORDER BY nam omogućava da uredimo podatke na izlazu
Primjer: ako imamo tabelu CHECKS

CHECKS#	PAYEE	AMOUNT	REMARKS
2	Reading R. R.	245.34	Train to Chicago
1	Cash	25	Wild Night Out
3	Joans Gas	25.1	Gas
4	Ma Bell	200.32	Cellular Phone

Da bi uredili ovu listu po broju čeka, upotrijebili bi sljedeću ORDER BY klauzulu:

SELECT*
FROM CHECKS
ORDER BY CHECK#;

CHECKS#	PAYEE	AMOUNT	REMARKS
1	Cash	25	Wild Night Out
2	Reading R. R.	245.34	Train to Chicago
3	Joans Gas	25.1	Gas
4	Ma Bell	200.32	Cellular Phone

13. Klauzula GROUP BY se ponaša isto kao i klauzula ORDER BY. Ova klauzula se koristi u kombinaciji sa agregatnim funkcijama. Klauzula GROUP BY pokreće agregatnu funkciju navedenu u SELECT iskazu za svaku grupu vrijednosti kolone koja je navedena u GROUP BY klauzuli.
14. HAVING klauzula nam omogućava da koristimo agregatne funkcije u iskazu poređenja, osiguravajući za agregatne funkcije ono što WHERE klauzula osigurava za pojedinačne redove.
15. Spajanje tabela možemo izvršiti na četiri načina. To su : spajanje po uslovu jednakosti, spajanje po bilo kom uslovu osim po jednakosti kolona, spoljašnje nasuprot unutrašnjeg spajanja i spajanje tabele sa samom sobom.
Spajanje po uslovu jednakosti se vrši u cilju pronalaženja istih vrijednosti kolone jedne tabele i kolone druge tabele.
Spajanje po bilo kom uslovu osim po jednakosti kolona kako sam naziv kaže da upiti spajanja po bilo kom uslovu koriste u WHERE klauzuli sve osim znaka =.
Spajanje tabela sa samom sobom obično se koristi za provjeru unutrašnje konzistentnosti podataka.

16. Podupit je upit čiji se rezultat prenosi kao argument drugom podupitu. Podupiti nam omogućavaju da povežemo nekoliko upita. Jednostavno rečeno podupit nam omogućava da vežemo skup rezultata jednog upita za drugi. Agregatne funkcije korištene u podupitima daju kao rezultat jednu vrijednost.
17. Ugnježdavanje (nesting) je čin umetanja podupita u drugi podupit. Dubina ugnježdavanja zavisi od implementacije SQL-a.
18. Korelisani podupiti koriste referisanje van podupita. Korelisan podupit se ponaša slično spajanju tabela. Korelacija se uspostavlja upotrebom elementa iz upita u podupitu. Podupit u korelisanom podupitu može dati rezultat koji se sastoji od više vrijednosti. Korelisani podupiti se takođe mogu koristiti u GROUP BY i HAVING klauzulama. Kada upotrebljavamo korelisane podupite, koji imaju GROUP BY i HAVING klauzule, klauzule u HAVING klauzuli moraju postojati ili u SELECT klauzuli ili u GROUP BY klauzuli. U suprotnom možemo dobiti poruku o grešci uz liniju: invalid column reference, zato što se podupit izvršava za svaku grupu a ne za slog. To znači da se ne može izvršiti valjano poređenje sa nekom veličinom ukoliko ona nije uključena prilikom formiranja grupe.

Primer 1.

Data je sledeća šema baze podataka $S = (S, I)$, pri čemu je skup šema relacija:

$S = \{ \text{Odeljenje}(\{SIF_OD, IME_OD, GRAD\}, \{SIF_OD\}), \text{Radnik}(\{SIF_RAD, IME, PREZIME, RADMES, SIF_RUK, DATZAP, PLATA, STIMUL, SIF_OD\}, \{SIF_RAD, SIF_OD\}) \}$,

a skup međurelacionih ograničenja:

$I = \{ \text{Radnik}[SIF_RUK] \subseteq \text{Radnik}[SIF_RAD], \text{Radnik}[SIF_OD] \subseteq \text{Odeljenje}[SIF_OD] \}$.

Obeležja šema relacija imaju sledeća značenja:

SIF_OD – šifra odeljenja,

$GRAD$ – lokacija odeljenja,

SIF_RAD – šifra (matični broj) radnika,

$RADMES$ – naziv radnog mesta ($\text{dom}(RADMES) = \{\text{analitičar, trg_putnik, službenik, prodavac, direktor}\}$)

SIF_RUK – šifra (matični broj) radnika koji je neposredni rukovodilac datom radniku

$DATZAP$ – datum zaposlenja

$PLATA$ – **mesečna** primanja radnika

$STIMUL$ – iznos stimulacije u dinarima za radnike koji su prodavci ili trgovački putnici

SIF_OD – šifra odeljenja u kome radnik radi.

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

Neka je $rbp = \{ radnik, odeljenje \}$ baza podataka nad šemom S .

Putem naredbi SQL-a realizovati sledeće upite.

(**NAPOMENA:** Upiti se realizuju bez zalaženja u sintaksne formalizme konkretnih softvera za rukovanje bazama podataka kao što su: PROGRESS, ORACLE, FoxPro. Na primer: oznaka kraja SQL naredbe u ORACLE softveru je simbol tačke-zareza (;), u PROGRESS-u tačka (.), u ACCESS-u simbol tačke-zareza (;), u Visual FoxPro softveru nema simbola za oznaku kraja SQL naredbe itd.)

PROSTI UPITI (PRETRAŽIVANJE BAZE PODATAKA)

1.1. Prikazati nazive i šifre odeljenja.

```
SELECT IME_OD, SIF_OD ODELJENJE  
FROM odeljenje
```

U ovom upitu se u listi atributa iza SELECT naredbe nalazi ime ODELJENJE. To je naziv atributa u rezultujućoj tabeli, koji će zameniti ime atributa IME_OD iz bazne relacije.

1.2. Prikazati sve podatke o radnicima koji rade u odeljenju sa šifrom 20.

```
SELECT *  
FROM radnik  
WHERE SIF_OD = 20
```

1.3. Prikazati imena i matične brojeve radnika i šifre odeljenja svih službenika.

```
SELECT IME, SIF_RAD, SIF_OD  
FROM radnik  
WHERE RADMES = "službenik"
```

Primena operatora: =, != (nije jednako ili <>), < (manje), <= (manje ili jednako), > (veće), >= (veće ili jednako), između dve vrednosti (BETWEEN ...AND...), lista vrednosti (IN *lista*), poređenje znakova LIKE (za tipove podataka *string*) i nula – vrednost (NULL), operator negacije NOT.

1.4. Prikazati imena odeljenja i njihove šifre i to za ona odeljenja, čije su šifre veće od 10.

```
SELECT IME_OD, SIF_OD  
FROM odeljenje  
WHERE SIF_OD > 10
```

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

1.5. Prikazati imena, plate i stimulacije za one radnike kod kojih je stimulacija veća od plate.

```
SELECT IME, PLATA, STIMUL
FROM radnik
WHERE STIMUL > PLATA
```

1.6. Prikazati imena, plate i šifre odeljenja za trgovačke putnike iz odeljenja 20, čija je plata veća ili jednaka od 400.

```
SELECT IME, PLATA, SIF_OD
FROM radnik
WHERE RADMES = "trg_putnik"
      AND SIF_OD = 20
      AND PLATA >= 400
```

Prava prenstva operatora

1.7. Prikazati podatke o ***svim*** analitičarima, iz ***svih*** odeljenja, i o svim službenicima odeljenja sa šifrom 10.

```
SELECT *
FROM radnik
WHERE RADMES = "analitičar"
      OR (RADMES = "službenik" AND SIF_OD = 10)
```

Zgrade u ovom upitu su nepotrebne, jer operator AND ima prvenstvo u odnosu na OR, ali je ovaj način pregledniji.

1.8. Prikazati ime, naziv radnog mesta i šifru odeljenja za upravnike koji ne rade u odeljenju 10.

```
SELECT IME, RADMES, SIF_OD
FROM radnik
WHERE RADMES = "upravnik"
      AND SIF_OD != 10
```

1.9. Prikazati sve podatke o radnicima koji nisu upravnici ili službenici a rade u odeljenju 20.

```
SELECT *
FROM radnik
WHERE NOT (RADMES = "upravnik" OR RADMES = "službenik")
      AND SIF_OD = 20
```

ili, upotrebom operatora IN:

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

```
SELECT *  
FROM radnik  
WHERE RADMES NOT IN ("upravnik", "službenik")  
AND SIF_OD = 20
```

1.10. Prikazati imena radnika koji imaju drugo slovo imena *a*.

```
SELECT IME  
FROM radnik  
WHERE IME LIKE "__a"
```

1.11. Prikazati imena, šifre odeljenja i plate radnika odeljenja 20 uređene u rastućem redosledu atributa PLATA.

```
SELECT IME, SIF_OD, PLATA  
FROM radnik  
WHERE SIF_OD = 20  
ORDER BY PLATA
```

SPOJ RELACIJA

1.12. Kako se zovu gradovi u kojima rade radnici sa imenom Mirko.

```
SELECT GRAD  
FROM radnik, odeljenje  
WHERE IME = "Mirko"  
AND radnik.SIF_OD = odeljenje.SIF_OD
```

ili:

```
SELECT GRAD  
FROM radnik r, odeljenje o  
WHERE IME = "Mirko"  
AND r.SIF_OD = o.SIF_OD
```

U ovim upitima u klauzuli WHERE u slučaju spoja relacija, koristimo ili puno ime relacije (*radnik, odeljenje*) ili sinonime (*r, o*), koji su navedeni u listi iza klauzule FROM. Inače, ovaj tip spoja se zove i ***SPOJ NA JEDNAKOST*** ili ***EQUI JOIN***, zato što je operator poređenja n-torki ovih dvaju relacija operator jednakosti (=).

SELF JOIN (SPAJANJE TABELE SAME SA SOBOM)

1.13. Prikazati imena i plate svih radnika, koji imaju platu veću od plate radnika Markovića.

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

```
SELECT r1.IME, r1.PLATA, r2.IME, r2.PLATA
FROM radnik r1, radnik r2
WHERE r1.PLATA > r2.PLATA
      AND r2.PREZIME = "Marković"
```

U ovom upitu, relacija *radnik* se spaja sama sa sobom na osnovu operatora > (veće): $r1.PLATA > r2.PLATA$. Svakoј n-torci relacije *r1* je pridružena n-torka sa vrednošću obeležja $PREZIME = "Marković"$ relacije *r2*, ako je zadovoljen uslov da je vrednost obeležja $PLATA$ n-torke relacije *r1* veća od vrednosti istog obeležja n-torke relacije *r2*.

Operator spoljnog spoja (+) OUTER JOIN

Da bi se u rezultat upita uključile i one n-torke (oni redovi) relacije koji ne zadovoljavaju uslov spajanja, koristi se operator + .

1.14. Prikazati podatke o odeljenjima, imena radnika koji u njima rade, ali u rezultat upita uključiti i podatke o odeljenjima, koji trenutno, možda, nemaju radnika.

```
SELECT o.SIF_OD, o.IME_OD, r.IME
FROM odeljenje o, radnik r
WHERE odeljenje.SIF_OD = radnik.SIF_OD (+)
ORDER BY SIF_OD
```

Simbol (+) u WHERE izrazu ima za posledicu da se relacija *radnik* posmatra kao da sadrži još jedan red (n-torku) više, koji sadrži nula-vrednosti za sve attribute. SQL spaja te redove tabele relacije *radnik* sa bilo kojim redom tabele *odeljenje*, koji se ne može spojiti sa stvarnim redom relacije *radnik*.

Upotreba *sinonima* relacija u SQL upitima omogućava spajanje relacija samih sa sobom, na način kao da je reč o dve posebne relacije. To se koristi naročito u slučajevima kada se želi spojiti (udružiti) jedan red tabele sa drugim iz iste tabele, odnosno, kada je u pitanju SELF JOIN.

1.15. Prikazati podatke o radnicima čija je plata veća od plate njihovih rukovodilaca.

```
SELECT osoba.IME, osoba.PLATA, upravnik.IME, upravnik.PLATA
FROM radnik osoba, radnik upravnik
WHERE osoba.SIF_RUK = upravnik.SIF_RAD
      AND osoba.PLATA > upravnik.PLATA
```

Rad sa numeričkim podacima

1.16. Prikazati ***ukupna*** primranja radnika, čije je radno mesto *trg_putnik*.

```
SELECT IME, PLATA, STIMUL, PLATA+STIMUL
FROM radnik
```

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

WHERE RADMES = "trg_putnik"

1.17. Prikazati podatke o radnicima čija je stimulacija veća od četvrtine njihovih plata.

```
SELECT IME, PLATA, STIMUL
FROM radnik
WHERE STIMUL > 0.25 * PLATA
```

Ovaj upit ilustruje situaciju u kojoj se u WHERE izrazu može koristiti *aritmetički izraz* ili, u ORDER BY klauzuli, što ilustruje sledeći upit.

1.18. Prikazati podatke o trgovačkim putnicima po opadajućem redosledu odnosa stimulacije i plata.

```
SELECT IME, STIMUL/PLATA, STIMUL, PLATA
FROM radnik
WHERE RADMES = "trg_putnik"
ORDER BY STIMUL/PLATA DESC
```

1.19. Prikazati *ukupna godišnja primanja* prodavaca.

```
SELECT IME, PLATA, STIMUL, 12 * (PLATA + STIMUL)
FROM radnik
WHERE RADMES = "prodavac"
```

1.20. Prikazati *dnevnu platu* radnika iz odeljenja 20 (uzeti da mesec ima 22 radna dana) i zaokružiti rezultat na dva decimalna mesta.

```
SELECT IME, PLATA, ROUND(PLATA/22,2)
FROM radnik
WHERE SIF_OD = 20
```

Ovaj upit ilustruje upotrebu funkcije ROUND, koja zaokružuje broj na naznačen broj decimalnih mesta.

Grupne funkcije (AVG(), COUNT(), MAX(), MIN(), SUM())

1.21. Prikazati *prosečnu* platu službenika.

```
SELECT AVG(PLATA)
FROM radnik
WHERE RADMES = "službenik"
```

1.22. Prikazati *prosečna godišnja* primanja trgovačkih putnika.

```
SELECT 12 * AVG( PLATA+STIMUL)
FROM radnik
WHERE RADMES = "trg_putnik"
```


SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

Funkcija COUNT() računa broj *numeričkih* vrednosti ili n-torki selektovanih pretraživanjem, koji imaju vrednost različitu od nule (od nula-vrednosti).

1.23. Koliko radnika prima stimulaciju?

```
SELECT COUNT(STIMUL)
FROM radnik
```

Da bi se eliminisale višetruke vrednosti atributa u n-torkama upotrebljava se izraz DISTINCT, odnosno upotrebom ove klauzule funkcija COUNT() će odrediti broj *različitih numeričkih* vrednosti.

1.24. Odrediti broj *različitih* radnih mesta u odeljenju 20.

```
SELECT COUNT(DISTINCT RADMEST)
FROM radnik
WHERE SIF_OD = 20
```

Oblik funkcije COUNT(*) određuje broj n-torki koji zadovoljavaju logički izraz u WHERE izrazu.

1.25. Koliko radnika radi u odeljenju 10?

```
SELECT COUNT(*)
FROM radnik
WHERE SIF_OD = 10
```

NAPOMENA: Ako se koristi **grupna funkcija** u SELECT naredbi, onda se ne može izdvojiti *pojedinačni* rezultat. To znači, na primer, da je pogrešna naredba:

```
SELECT IME, AVG(PLATA)
```

Atribut IME ima vrednost za *svaku* n-torku, dok AVG(PLATA) daje kao rezultat *jednu* vrednost za ceo upit. Kao ilustracija ovog slučaja, poslužiće sledeći upit.

1.26. Koji radnik ima najveću platu?

```
SELECT IME, PLATA
FROM radnik
WHERE PLATA = (SELECT MAX(PLATA)
               FROM radnik)
```

1.27. Kolika su ukupna primanja trgovačkih putnika?

```
SELECT IME, PLATA, STIMUL, PLATA+STIMUL
```

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

```
FROM radnik  
WHERE RADMES = "trg_putnik"
```

GROUP BY klauzula

1.28. Izračunati prosečnu platu za **svako** odeljenje.

```
SELECT SIF_OD, AVG(PLATA)  
FROM radnik  
GROUP BY SIF_OD
```

1.29. Izračunati prosečnu **godišnju** platu za svako odeljenje, izuzimajući upravnike i direktora.

```
SELECT SIF_OD, 12 * AVG(PLATA)  
FROM radnik  
WHERE RADMES NOT IN ("upravnik", "direktor")  
GROUP BY SIF_OD
```

N-torke tabele moguće je grupisati i po više atributa. To ilustruje sledeći primer.

1.30. Izračunati **prosečnu** platu za **svako** radno mesto grupisano po odeljenju.

```
SELECT SIF_OD, RADMES, COUNT(*), 12 * AVG(PLATA)  
FROM radnik  
GROUP BY SIF_OD, RADMES
```

HAVING klauzula

Putem ove klauzule moguće je selektovati **grupe** n-torki, pošto su te grupe već formirane sa GROUP BY klauzulom.

1.31. Prikazati prosečnu platu za sve grupe radnih mesta, na kojima radi više od 2 radnika.

```
SELECT RADMES, COUNT(*), AVG(PLATA)  
FROM radnik  
GROUP BY RADMES  
HAVING COUNT(*) > 2
```

1.32. Prikazati nazive radnih mesta, za koje je prosečna plata veća od upravnikove plate.

```
SELECT RADMES, AVG(PLATA)  
FROM radnik  
GROUP BY RADMES
```

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

```
HAVING AVG(PLATA) >
      (SELECT AVG(PLATA)
       FROM radnik
       WHERE RADMES = "upravnik")
```

Rad sa nula-vrednostima (funkcija NVL())

Numerička vrednost atributa **nula** u relaciji je semantički drugačija od one vrednosti koja sadrži **broj nula (0)**. Vrednost **nula** ili null-vrednost se prikazuje kao prazan prostor, a broj nula kao broj (0). Na primer, samo radnici koji su raspoređeni na radno mesto *trgovačkih putnika* ili *prodavaca* imaju stimulaciju, pa je za te n-torke vrednost atributa STIMUL relacije *radnik* definisana (eventualno za one trgovačke putnike ili prodavce, koji nisu dobili stimulaciju, jer nisu ispunili neke od uslova za to). Za sve ostale n-torke relacije *radnik* vrednost atributa STIMUL je neodređena (odn. neprimenjivo svojstvo), odnosno, kažemo da je to nula-vrednost (NULL).

1.33. Prikazati podatke o radnicima koji nemaju stimulaciju.

```
SELECT IME, PLATA, STIMUL, RADMES
FROM radnik
WHERE STIMUL IS NULL
```

1.34. Koliko ima radnika u odeljenju 10 koji primaju platu i stimulaciju.

```
SELECT COUNT(PLATA), COUNT(STIMUL)
FROM radnik
WHERE SIF_OD = 10
```

U slučaju da je potrebno sabrati vrednosti PLATA + STIMUL i to za sve n-torke, potrebno je nula-vrednost zameniti nekom određenom numeričkom vrednošću. Za to se koristi **funkcija NVL()**.

1.35. Prikazati **ukupna** primanja za **svakog** radnika posebno.

```
SELECT IME, PLATA, STIMUL, PLATA + NVL(STIMUL,0)
FROM radnik
```

U ovom upitu, sve nula-vrednosti atributa STIMUL u relaciji *radnik* biće zamenjene numeričkom vrednošću 0.

1.36. Izračunati prosečnu stimulaciju, prosečna primanja, kao i ukupna primanja za **celo** preduzeće.

```
SELECT AVG(STIMUL), AVG(PLATA + NVL(STIMUL,0)),
      SUM(PLATA + NVL(STIMUL,0))
FROM radnik
```

PODUPITI

1.37. Prikazati imena i nazive radnih mesta radnika koji imaju isto radno mesto kao i radnik Marković.

```
SELECT IME, RADMES
FROM radnik
WHERE RADMES =
      (SELECT RADMES
       FROM radnik
       WHERE PREZIME = "Marković")
```

Naredbe koje podržavaju WHERE izraz su: SELECT, UPDATE, DELETE, INSERT i CREATE.

1.38. Koji radnici zarađuju više od ***bilo kog*** radnika odeljenja 10.

```
SELECT IME, PLATA, SIF_OD
FROM radnik
WHERE PLATA > ANY
      (SELECT PLATA
       FROM radnik
       WHERE SIF_OD = 10)
```

Ako u prethodnom upitu umesto operatora ANY upotrebimo operator ALL, pretraživanje će selektovati n-torke čija vrednost atributa PLATA je veća od ***svih*** vrednosti dobijenih podupitom.

1.39. Prikazati podatke o radnicima iz odeljenja 10 sa istim radnim mestom kao ***bilo ko*** iz odeljenja 20.

```
SELECT IME, RADMES
FROM radnik
WHERE SIF_OD = 10
      AND RADMES IN
      (SELECT RADMES
       FROM RADNIK
       WHERE SIF_OD = 20)
```

U slučaju ORACLE softvera moguće je upoređivati više atributa u spoljašnjem i unutrašnjem upitu. Na primer, kao u sledećem upitu.

1.40. Prikazati podatke o radnicima koji imaju isto radno mesto i platu kao analitičar Jović.

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

```
SELECT IME, RADMES, PLATA
FROM radnik
WHERE (RADMES, PLATA) =
      (SELECT RADMES, PLATA
       FROM radnik
       WHERE PREZIME = "Jović")
```

Mogući su i takvi oblici upita koji sadrže više uslova sa podupitima, koji su spojeni operatorima AND i OR.

1.41. Prikazati podatke o radnicima koji imaju isto radno mesto kao i Marković, ili istu ili veću platu kao Jović, i to uređene prema nazivima radnih mesta, a zatim prema rastućem redosledu plata.

```
SELECT IME, RADMES, SIF_OD, PLATA
FROM radnik
WHERE RADMES =
      (SELECT RADMES
       FROM radnik
       WHERE PREZIME = "Marković")
OR
      PLATA > =
      (SELECT PLATA
       FROM radnik
       WHERE PREZIME = "Jović")
ORDER BY RADMES, PLATA
```

Takođe, podupit može sadržati podupite (odn. ugnježdene podupite).

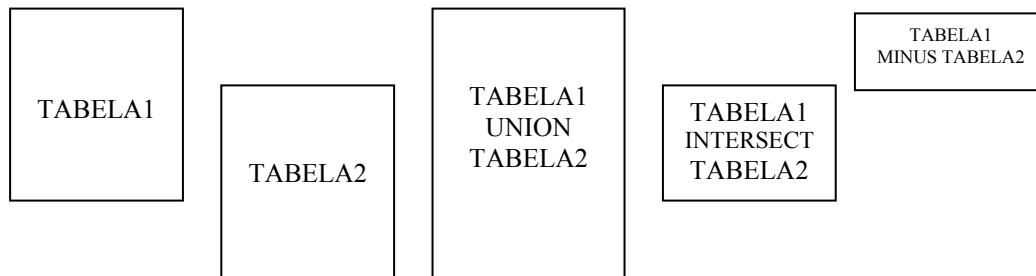
1.42. Prikazati podatke o radnicima odeljenja 20 sa istim radnim mestom kao bilo ko iz odeljenja ANALIZA.

```
SELECT IME, RADMES
FROM radnik
WHERE SIF_OD = 20
      AND RADMES IN
      (SELECT RADMES
       FROM radnik
       WHERE SIF_OD =
            (SELECT SIF_OD
             FROM odeljenje
             WHERE IME_OD = "ANALIZA"))
```

Primena operatora unije (UNION), preseka (INTERSECT) i razlike (MINUS)

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

Upit (spoljašnji ili unutrašnji) može biti sastavljen od dva ili više SELECT blokova, koji su povezani operatorima (UNION, INTERSECT, MINUS). To ilustruje sledeća slika (Marjanović Zoran, *ORACLE relacioni SUBP*, Beograd, 1990, str. 31).



1.43. Prikazati podatke o radnicima čija je plata jednaka plati koju ima radnik Marković ili Jović.

```
SELECT IME, PLATA
FROM radnik
WHERE PLATA IN
    (SELECT PLATA
     FROM radnik
     WHERE PREZIME = "Marković"
    UNION
    SELECT PLATA
     FROM radnik
     WHERE PREZIME = "Vasić")
```

Podupit može davati za rezultat informacije iz više relacija. To ilustruje sledeći primer.

1.44. Koji radnici obavljaju isti posao kao radnici koji rade u odeljenjima čija je lokacija Novi Sad.

```
SELECT IME, PREZIME, RADMES
FROM radnik
WHERE RADMES IN
    (SELECT RADMES
     FROM radnik, odeljenje
     WHERE odeljenje.GRAD = "Novi Sad"
     AND radnik.SIF_OD = odeljenje.SIF_OD)
```

S obzirom da se informacije o lokaciji odeljenja u kojima su zaposleni radnici nalaze u relaciji *odeljenje*, u podupitu je potrebno spojiti relacije *radnik* i *odeljenje* na osnovu istih vrednosti zajedničkih obeležja ovih relacija a to je obeležje SIF_OD.

Do sada su bili navedeni primeri upita, kod kojih se podupit izvršavao samo *jednom*, a rezultat podupita se koristi u WHERE izrazu glavnog upita. Međutim, postoje i takvi upiti, kod kojih se podupiti izvršavaju *više puta*, jednom za svaku n-torku koja se koristi u glavnom upitu. To su **korelisani upiti** ili **zavisni ugdnježdeni upiti** (Mogin P, Luković I, *Principi baza podataka*, Novi Sad, 1996, str. 209)

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

1.45. Koji radnici zarađuju više od prosečne plate za odeljenje u kome rade?

U ovom upitu glavni upiti će izgledati:

```
SELECT SIF_OD, IME, PLATA
```

```
FROM radnik
```

```
WHERE PLATA > prosečne plate za odeljenje u kome radnik radi
```

Podupit će izgledati:

```
(SELECT AVG(PLATA)
```

```
FROM radnik
```

```
WHERE SIF_OD = šifra odeljenja za datog radnika)
```

Sada je jasno zašto se ovakvi upiti zovu ***zavisni ugnježdeni upiti***. Izvršavanje podupita zavisi od rezultata spoljašnjeg (glavnog) upita. U ovom slučaju, šifra odeljenja u podupitu zavisi od šifre odeljenja, koja je selektovana u spoljašnjem upitu.

Upit ima konačni izgled:

```
SELECT SIF_OD, IME, PLATA
```

```
FROM radnik rX
```

```
WHERE PLATA >
```

```
    (SELECT AVG(PLATA)
```

```
    FROM radnik rY
```

```
    WHERE rX.SIF_OD = rY.SIF_OD)
```

Upotreba operatora EXISTS

U slučaju upotrebe operatora EXISTS glavni upit će se izvršiti ako unutrašnji upit kao rezultat pretraživanja ima bar jednu n-torku. (Opširnije o ovom operatoru u primeru br. 3).

1.46. Prikazati podatke o radnicima koji imaju najmanje jednog radnika koji im je podređen.

```
SELECT IME, PREZIME, RADMES, SIF_OD
```

```
FROM radnik rX
```

```
WHERE EXISTS
```

```
    (SELECT *
```

```
    FROM radnik rY
```

```
    WHERE rX.SIF_RAD = rY.SIF_RUK)
```

Ažuriranje baze podataka (naredbe INSERT, UPDATE i DELETE)

1.47. Uneti podatke o novom radniku.

```
INSERT INTO radnik
```

```
VALUES(7978, "Zdravko", "Čolić", "analitičar", 7790, "01.05.99", 1000, NULL, 20)
```

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

S obzirom da se radi o radniku koji obavlja posao *analitičara*, pa nema pravo na stimulaciju, vrednost atributa STIMUL je nula-vrednost, odnosno, NULL (po sintaksi ORACLE softvera).

U naredbi INSERT može se koristiti i podupit, pa se na taj način mogu n-torke iz jedne relacije uneti u drugu. Podupit zamenjuje izraz VALUES.

1.48. Svim analitičarima i upravicima dati premiju u iznosu od 10% njihovih plata. Te informacije uneti u relaciju *premiya* zajedno sa datumom zaposlenja.

```
INSERT INTO PREMIJA(SIF_RAD,PREMIJA, RADMES, DATZAP)
  SELECT SIF_RAD, 0.10 * PLATA, RADMES, DATZAP
  FROM radnik
  WHERE RADMES IN ("analitičar", "upravnik")
```

1.49. Svim trgovačkim putnicima i savetnicima iz odeljenja 10 povećati platu za 8%.

```
UPDATE radnik
  SET PLATA = PLATA * 1.08
  WHERE (RADMES = "trg_putnik" OR RADMES = "savetnik")
  AND SIF_OD = 10
```

Naredba UPDATE može da koristi i podupite u WHERE klauzuli.

1.50. Sve radnike koji su zaposleni u Novom Sadu rasporediti na radno mesto službenika.

```
UPDATE radnik
  SET RADMES = "službenik"
  WHERE SIF_OD IN
    (SELECT SIF_OD
     FROM odeljenje
     WHERE GRAD = "Novi Sad")
```

1.51. Izbrisati podatke o svim radnicima koji rade u odeljenju PRIPREMA.

```
DELETE radnik
WHERE SIF_OD IN (SELECT SIF_OD
                 FROM odeljenje
                 WHERE IME_OD = "PRIPREMA")
```

Definisanje fizičke strukture baze podataka putem jezika SQL

1.52. Kreirati tabele *odeljenje* i *radnik*.

```
CREATE TABLE odeljenje
  (SIF_OD NUMBER(2) NOT NULL,
   IME_OD CHAR(15),
```


GRAD CHAR(20))

```
CREATE TABLE radnik
(SIF_RAD NUMBER(6) NOT NULL,
 IME CHAR(15),
 PREZIME CHAR(20),
 RADMES CHAR(25),
 SIF_RUK NUMBER(6),
 DATZAP DATE,
 PLATA NUMBER(6),
 STIMUL NUMBER(5),
 SIF_OD NUMBER(2) NOT NULL)
```

Na ovom mestu treba ukazati na nedostatak naredbe CREATE TABLE, koji se sastoji u nemogućnosti definisanja *primarnog ključa relacije*, a to je jedno od osnovnih osobina relacionog modela. Klauzula NOT NULL nije dovoljna, jer se ona može koristiti za bilo koji atribut relacije. Zato se formiraju stabla pristupa (indeksi), zajedno sa klauzulom NOT NULL. Osim što omogućavaju implementaciju integriteta entiteta, stabla pristupa obezbeđuju efikasno traženje i pretraživanje podataka tabele. Klauzula UNIQUE ukazuje da će indeks, a samim tim i tabela, sadržati *jedinstvene vrednosti* nad navedenim nizom obeležja. (Mogin P, Luković I, *Principi baza podataka*, Novi Sad, 1996., str. 215).

1.53. Kreirati jedinstveni indeks nad relacijom *radnik* za attribute SIF_RAD i SIF_OD (ovi atributi čine primarni ključ šeme relacije *Radnik*).

```
CREATE UNIQUE INDEX idx_radnik
ON radnik (SIF_RAD, SIF_OD)
```

1.54. Kreirati jedinstveni indeks nad relacijom *odeljenje* za atribut SIF_OD (ovaj atribut je primarni ključ šeme relacije *Odeljenje*).

```
CREATE UNIQUE INDEX idx_odeljenje
ON odeljenje (SIF_OD ASC)
```

Evo nekoliko interesantnih informacija u vezi sa kreiranjem indeksa nad tabelama baze podataka. Tekst je preuzet u nešto izmenjenom i skraćenom obliku iz uputstva PROGRESS relacionog SRBP-a (*Defining Indexes, PROGRESS Database Design Guide Str. 4 – 4*). Inače, ova zapažanja odnose se, uglavnom, i na sve relacione softvere za rukovanje bazom podataka.

Zašto definisati Indekse?

- Direktni pristup i brzo pretraživanje zapisa (n-torki). Ako se vrši pretraživanje u bazi, PROGRESS neće ići u glavnu tabelu, Umesto toga, on koristi **direktni** pristup po indeksu odn. koristi pokazivač da bi pročitao odgovarajući slog (tj. zapis) u tabeli.

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

- Automatski redosled slogova.

Indeks automatski sortira slogove sekvencijalno (umesto redosleda kojim se slogovi kreiraju i memorišu na disku).

- Omogućava jedinstvenost.

Ako se definiše jedinstveni (*unique*) indeks za tabelu, sistem obezbeđuje da se ne smeju uneti dva sloga sa istom vrednošću za taj indeks.

- Rapidno procesiranje veza između tabela.

Dve tabele su povezane ako se definiše polje (ili više polja) u jednoj tabeli, koje se koristi za pristup slogovima u drugoj tabeli. Ako tabela, kojoj se pristupa, ima indeks, koji je baziran na odgovarajućem polju, onda je pristup slogovima efikasniji.

Primetimo da polje, koje se koristi za povezivanje dvaju tabela, ne mora imati isto ime u obe tabele.

Na primer:

```
FOR EACH narudžba WHERE  
    narudžba.id_kupca = kupac.id_kupca
```

Ove naredbe kažu PROGRESS-u da pronade sve narudžbe za određenog kupca. PROGRESS će efikasno izvršiti ovaj kod, ako postoji indeks u tabeli *Narudžba*, koji je baziran na indeksu *id_kupca*, ili ako je *id_kupca* prvo polje u indeksu, koji ima više polja.

Obe naredbe, FOR EACH i FIND, koriste WHERE opciju da bi se locirale na jednog ili više povezanih slogova. Polja, koja se koriste u WHERE opciji, su indeksirana, što znači da PROGRESS može da pronade odgovarajuće slogove bez skeniranja cele tabele.

Nedostaci definisanja Indeksa

- Indeksi zauzimaju mnogo mesta na disku.
- Indeksi mogu da uspore druge procese. Kada korisnik menja indeksirano polje, PROGRESS automatski menja sve povezane indekse. Takođe, kada korisnik kreira ili briše slog, PROGRESS menja sve indekse za tu tabelu.

Definišite indekse koje vaša aplikacija zahteva, ali izbegnite indekse koji omogućavaju smanjenje koristi ili koji se često koriste. Na primer, umesto da prikazujete podatke u posebnim redosledima, bolje je da sortirate podatke kada ih i prikazujete, nego da definišete indeks za automatsko sortiranje.

Izbor Tabela i Polja za Indeksiranje

Ako izvodite često dodavanje, brisanje i izmene nad malom tabelom, možete izbeći indeks, zbog toga što usporava performanse izazvane prekoračenjem indeksa. Međutim,

ako često vršite upite, onda je korisno kreirati indeks za tabelu. Možete indeksirati polje po kome se često vrši pretraga, i to po redosledu, po kome se vrši pretraga.

Nemojte kreirati indeks ako vršite pretragu nad velikim procetnom slogova u bazi (na primer, 19.000 od ukupno 20.000 slogova). Efikasnije je skenirati tabelu. Međutim, isplati se da kreirate indeks za upite nad malim brojem slogova (na primer, 100 od 20.000). Na ovaj način, PROGRESS skenira samo tabelu indeksa umesto cele tabele.

Indeksi i Record ID

Indeks je lista vrednosti indeksa i ID-ova slogova (*RECIDs*). *RECIDs* je fizički pokazivač na tabele baze podataka koji vam omogućuje brzi pristup slogovima.

Baza se sastoji od blokova baze. Blok baze podataka može da sadrži 512, 1024 ili 2048 bajtova podataka, zavisno od operativnog sistema. Blokovi se koriste za memorisanje slogova baze i indeksiranje. Višestruki indeksi se memorišu u blokovima baze. Blok baze indeksa je organizovan u strukturu tipa stabla za brži pristup. PROGRESS locira slogove putem indeksa obilaskom po stablu indeksa. Kada je slog lociran, ID sloga pristupa podatku. PROGRESS ne zaključava celo stablo indeksa dok vrši traženje sloga. On samo zaključava blok, koji sadrži slog. Međutim, drugi korisnici mogu simultano da pristupaju slogovima u istoj bazi. □

Izmena strukture tabele (relacije)

1.55. Dodati novo obeležje GRAD_STAN (grad u kome stanuje radnik) u relaciju *radnik*.

```
ALTER TABLE radnik  
ADD (GRAD_STAN CHAR(20))
```

1.56. Promeniti širinu atributa PLATA na 8 da bi se dodala 2 decimalna mesta.

```
ALTER TABLE radnik  
MODIFY (PLATA NUMBER(8.2))
```

Izbacivanje relacije iz baze podataka DROP TABLE

1.57. Izbaciti definiciju tabele PREMIJA iz baze podataka.

```
DROP TABLE premija
```

Postoji važna razlika između naredbi DROP TABLE i DELETE. Naredbom DELETE se može brisati kompletan sadržaj tabele, ali sama tabela ostaje u rečniku baze podataka i sa njom se može i dalje raditi. Naredbom DROP TABLE *ime_tabele*, tabela sa sadržajem se izbacuje iz baze podataka i više nije dostupna.

Pogledi (izvedene ili virtuelne relacije) – VIEW

Pogled je virtuelna relacija – ona ne postoji fizički na disku, odnosno, ne zauzima nikakav memorijski prostor, ali se sa njom može raditi kao i sa svakom drugom relacijom.

Pogledi se koriste iz sledećih razloga:

- u slučaju da je potrebno neke podatke u bazi zaštititi od neovlašćenog pristupanja, kreira se pogled na tabelu sa željenim (svima dostupnim) informacijama (atributima),
- pogledi zahtevaju korišćenje jednostavnijih pretraživanja, odnosno, ubrzavaju performanse obrade.

1.58. Kreirati pogled koji će se sastojati samo od podataka o zaposlenima u odeljenju 10.

```
CREATE VIEW ODELJENJE10 AS
SELECT SIF_RAD, IME, PREZIME, SIF_RUK
FROM radnik
WHERE SIF_OD = 10
```

1.59. Kreirati pogled koji će davati informacije o platama i premijama radnika bez navođenja imena radnika.

```
SELECT VIEW ZARADA AS
SELECT SIF_RAD, SIF_OD, PLATA, PREMIJA
FROM radnik
```

1.60. Kreirati pogled sa atributima: šifra odeljenja, posao, ukupna i srednja plata radnika na određenim radnim mestima i to za svako odeljenje.

```
CREATE VIEW FINANSIJE (SIF_OD, RADMES, UKUPNA_PLATA, PROS_PLT)AS
SELECT SIF_OD, RADMES, SUM(PLATA), AVG(PLATA)
FROM radnik
GROUP BY SIF_OD, RADMES
```

Sada se relacija *finansije* može koristiti kao i svaka druga bazna relacija. Na primer:

```
SELECT *
FROM finansije
```

Primer 2.

Data je sledeća šema baze podataka $S = (S, I)$, pri čemu je skup šema relacija:

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

$S = \{ \text{Student}(\{BR_IND, IME, PREZIME, DATUM_ROD, PTT_STAN, SMER\}, \{BR_IND\}),$
 $\text{Grad}(\{PTT, GRAD\}, \{PTT\}),$
 $\text{Nastavnik}(\{SIF_NAST, IME_NAS, PREZ_NAS, DAT_ROD, PTT_STAN, ZVANJE, SIF_KAT, PLT, SIF_RUKOV\}, \{SIF_NAST\}),$
 $\text{Katedra}(\{SIF_KAT, NAZIV_KAT\}, \{SIF_KAT\}),$
 $\text{Predmet}(\{SIF_PRED, NAZIV_PRED\}, \{SIF_PRED\}),$
 $\text{Predaje}(\{SIF_PRED, SIF_NAST, FOND\}, \{SIF_PRED, SIF_NAST\})$
 $\text{Polaže}(\{BR_IND, SIF_PRED, DATUM, OCENA\}, \{BR_IND, SIF_PRED\} \quad \},$

a skup međurelacionih ograničenja:

$I = \{ \text{Polaže}[BR_IND] \subseteq \text{Student}[BR_IND],$
 $\text{Student}[PTT_STAN] \subseteq \text{Grad}[PTT],$
 $\text{Predaje}[SIF_NAST] \subseteq \text{Nastavnik}[SIF_NAST],$
 $\text{Nastavnik}[PTT_STAN] \subseteq \text{Grad}[PTT],$
 $\text{Nastavnik}[SIF_RUKOV] \subseteq \text{Nastavnik}[SIF_NAST],$
 $\text{Nastavnik}[SIF_KAT] \subseteq \text{Katedra}[SIF_KAT],$
 $\text{Predaje}[SIF_PRED] \subseteq \text{Predmet}[SIF_PRED],$
 $\text{Polaže}[SIF_PRED] \subseteq \text{Predmet}[SIF_PRED] \}.$

Obeležja šema relacija imaju sledeća značenja:

BR_IND – broj indeksa studenta,

PTT_STAN – poštanski broj mesta u kome student (nastavnik) stanuje,

SMER – šifra smeru koga student pohađa,

PLT – mesečna plata nastavnika

SIF_RUKOV – matični broj nastavnika koji je rukovodilac datog nastavnika.

Neka je $rbp = \{ student, grad, nastavnik, katedra, predmet, predaje, polaže \}$ baza podataka nad šemom S .

Putem naredbi SQL-a realizovati sledeće upite (bez zalaženja u sintaksne formalizme konkretnih softvera za rukovanje bazama podataka (PROGRESS, ORACLE, FoxPro)).

2.1. Prikazati imena, prezimena i brojeve indeksa **svih** studenata.

```
SELECT IME, PREZIME, BR_IND
FROM student
```

2.2. Prikazati samo **različita** imena i prezimena svih studenata.

```
SELECT DISTINCT IME, PREZIME
```

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

FROM student

2.3. Prikazati imena i prezimena nastavnika, koji zarađuju više od 600 dinara i koji nemaju svog rukovodioca.

```
SELECT IME_NAS, PREZ_NAS
FROM nastavnik
WHERE PLT > 600 AND SIF_RUKOV IS NULL
```

Podizraz `SIF_RUKOV IS NULL` obezbeđuje selektovanje samo onih torki tabele *nastavnik*, za koje je vrednost torke nad obeležjem *SIF_RUKOV* nula-vrednost.□

2.4. Prikazati brojeve indeksa, imena i prezimena svih studenata, koji su rođeni između 01.01.1980 i 01.01.1981. (uključujući i granične vrednosti) i čije prezime počinje sa slovom N.

```
SELECT BR_IND, IME, PREZIME
FROM student
WHERE (DATUM_ROD BETWEEN "01.01.1980" AND "01.01.1981") AND
(PREZIME LIKE 'N%')
```

2.5. Prikazati šifre nastavnika, imena, prezimena i **godišnje** plate svih nastavnika, zaokružene na dva decimalna mesta, za koje polovina mesečne zarade nije manja od 300 dinara.

```
SELECT SIF_NAST, IME_NAS, PREZ_NAS, ROUND(12*PLT,2)
FROM nastavnik
WHERE NOT (0.5*PLT < 300)
```

2.6. Prikazati imena, prezimena i šifre smerova za one studente, koji pohađaju smerove 021, 022 i 023 ili su iz Zrenjanina.

```
SELECT IME, PREZIME, SMER
FROM student
WHERE SMER IN (021, 022, 023) OR PTT_STAN = "23000"
```

2.7. Prikazati imena i prezimena nastavnika koji su u zvanju vanrednih ili redovnih profesora.

```
SELECT IME_NAS, PREZ_NAS, ZVANJE
FROM nastavnik
WHERE ZVANJE IN ('vanredni profesor', 'redovni profesor')
```

ili:

```
SELECT IME_NAS, PREZ_NAS, ZVANJE
FROM nastavnik
```

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

WHERE (ZVANJE ='vanredni profesor' OR ZVANJE = 'redovni profesor').

2.8. Prikazati imena i prezimena nastavnika koji su u zvanju docenta i koji su iz Novog Sada.

```
SELECT IME_NAS, PREZ_NAS
FROM nastavnik
WHERE ZVANJE ='docent' AND PTT_STAN = 021.
```

2.9. Koji studenti su stariji od 20. godina? (Trenutno je 1999. godina)

```
SELECT *;
FROM student
WHERE student.DATUM_ROD < CTOD("01.01.79")
```

NAPOMENA: Ovaj upit može da ima različite oblike u zavisnosti od konkretnog softvera za rukovanje bazama podataka, odnosno, od načina putem koga se vrši upoređivanje različitih tipova podataka (string, numerički, datumski, logički tipovi podataka). U ovom upitu se koristi sintaksa FoxPro softvera, po kojoj se putem funkcije CTOD() niz karaktera pretvara u datumski tip podatka, s obzirom da je atribut DATUM_ROD relacije *student* definisan kao datumski tip podatka.

Uređivanje izlaznih rezultata upita

2.10. Prikazati podatke o nastavnicima, koji imaju svog rukovodioca, saglasno rastućem redosledu šifre rukovodioca, zatim, saglasno rastućem redosledu prezimena nastavnika (unutar redosleda po šifri rukovodioca), a na kraju po opadajućem redosledu imena nastavnika.

```
SELECT SIF_NAST, IME_NAS, PREZ_NAS, SIF_RUKOV
FROM nastavnik
WHERE SIF_RUKOV IS NOT NULL
ORDER BY SIF_RUKOV ASC, PREZ_NAS ASC, IME_NAS DESC
```

Upotreba skupovnih funkcija

2.11. Prikazati ukupan broj nastavnika, srednju vrednost plate svih nastavnika i ukupan godišnji iznos plata svih nastavnika.

```
SELECT COUNT(*), AVG(PLT), 12* SUM(PLT)
FROM nastavnik
```

2.12. Prikazati ukupan broj nastavnika koji imaju svog rukovodioca.

```
SELECT COUNT(SIF_RUKOV)
```

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

FROM nastavnik

2.13. Prikazati ukupan broj rukovodilaca nastavnika.

```
SELECT COUNT(DISTINCT SIF_RUKOV)
FROM nastavnik
```

2.14. Prikazati minimalnu i maksimalnu platu nastavnika.

```
SELECT MIN(PLT), MAX(PLT)
FROM nastavnik
```

Spajanje tabela

2.15. Izlistati nazive katedri, imena i prezimena nastavnika koji pripadaju tim katedrama.

```
SELECT katedra.NAZIV_KAT, nastavnik.IME_NAS, nastavnik.PREZ_NAS
FROM katedra, nastavnik
WHERE katedra.SIF_KAT = nastavnik.SIF_KAT
ORDER BY katedra.NAZIV_KAT
```

2.16. Izlistati imena i prezimena nastavnika, kao i nazive predmeta koje oni predaju i u kom fondu časova.

```
SELECT nastavnik.IME_NAS, nastavnik.PREZ_NAS, predmet.NAZIV_PRED,
       predaje.FOND
FROM nastavnik, predaje, predmet
WHERE predmet.SIF_PRED = predaje.SIF_PRED
      AND nastavnik.SIF_NAST = predaje.SIF_NAST
ORDER BY nastavnik.PREZ_NAS
```

2.17. Prikazati imena i prezimena studenata, kao i imena gradova gde studenti stanuju.

```
SELECT IME, PREZIME, GRAD
FROM student, grad
WHERE student.PTT_STAN = grad.PTT
```

2.18. Izlistati imena i prezimena studenata, kao i imena nastavnika, kod koga su studenti polagali date ispite, kao i imena katedri kojima nastavnici pripadaju.

```
SELECT student.IME, student.PREZIME, nastavnik.IME_NAS, katedra.NAZIV_KAT
FROM student, polaže, nastavnik, katedra
WHERE student.BR_IND = polaže.BR_IND
      AND polaže.SIF_PRED = predmet.SIF_PRED
      AND predaje.SIF_NAST = nastavnik.SIF_NAST
      AND katedra.SIF_KAT = Nastavnik.SIF_KAT
```


SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

2.19. Izlistati imena studenata, nazive predmeta koje su položili i sa kojom ocenom i imena nastavnika koji su ih ocenili (ispitivali).

```
SELECT student.IME, student.PREZIME, predmet.NAZIV_PRED, polaže.OCENA
       nastavnik.IME_NAS,
FROM student, polaže, predmet, nastavnik
WHERE student.BR_IND = polaže.BR_IND
      AND polaže.SIF_PRED = predmet.SIF_PRED
      AND predaje.SIF_NAST = nastavnik.SIF_NAST
```

Grupisanje podataka

2.19. Koliko studenata ima **svaki** smer?

```
SELECT COUNT(*), SMER
FROM student
GROUP BY SMER
```

2.20. Kolika je prosečna mesečna plata za **svako** zvanje nastavnika.

```
SELECT nastavnik.ZVANJE, AVG(PLT)
FROM nastavnik
GROUP BY zvanje.
```

2.21. Koliko ima studenata u **svakom** od gradova ?

```
SELECT student.PTT_STAN, grad.GRAD, COUNT(*)
FROM student, grad
WHERE student.PTT_STAN = grad.PTT
GROUP BY student.PTT_STAN, grad.GRAD
```

2.22. Koja je prosečna ocena za **svaki** predmet?

```
SELECT predmet.NAZIV_PRED, polaže.AVG(OCENA)
FROM predmet, polaže
WHERE predmet.SIF_PRED = polaže.SIF_PRED
GROUP BY polaže.SIF_PRED, predmet.NAZIV_PRED
```

2.23. Prikazati nazive predmeta koje je položilo više od 10 studenata.

```
SELECT predmet.NAZIV_PRED, COUNT(*)
FROM predmet, polaže
WHERE predmet.SIF_PRED = polaže.SIF_PRED
GROUP BY polaže.SIF_PRED, predmet.NAZIV_PRED
HAVING COUNT(*) > 10.
```

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

2.24. Prikazati brojeve indeksa, imena i prezimena studenata koji su položili više od 6 predmeta.

```
SELECT student.BR_IND, student.IME, student.PREZIME, COUNT(*)
FROM student, polaže
WHERE student.BR_IND = polaže.BR_IND
GROUP BY polaže.BR_IND, student.IME, student.PREZIME
HAVING COUNT(*) > 6.
```

2.25. Prikazati nazive katedri na kojima radi više od 5 nastavnika.

```
SELECT katedra.NAZIV_KAT, COUNT(*)
FROM katedra, nastavnik
WHERE katedra.SIF_KAT = nastavnik.SIF_KAT
GROUP BY katedra.NAZIV_KAT
HAVING COUNT(*) > 5.
```

Ugnježdjeni upiti

2.26. Prikazati imena, prezimena i plate nastavnika koji zarađuju više od prosečne plate.

```
SELECT IME_NAS, PREZ_NAS, PLT
FROM nastavnik
WHERE PLT > ANY (SELECT AVG(PLT) FROM nastavnik)
```

(U ovom slučaju, kada ugnježdjeni podupit kao rezultat daje tačno jednu vrednost, tada se službena reč ANY može izostaviti.)

2.27. Prikazati imena i prezimena nastavnika koji ne predaju nijedan predmet.

```
SELECT nastavnik.IME_NAS, nastavnik.PREZ_NAS
FROM nastavnik
WHERE nastavnik.SIF_NAST NOT IN (SELECT DISTINCT predaje.SIF_NAST
                                FROM predaje)
```

2.28. Prikazati nazive predmeta koje nije položio nijedan student.

```
SELECT predmet.NAZIV_PRED
FROM predmet
WHERE predmet.SIF_PRED NOT IN (SELECT DISTINCT polaže.SIF_PRED
                                FROM polaže)
```

NAPOMENA: Predikatski izraz =ANY ekvivalentan je izrazu IN, dok je izraz !=ALL ekvivalentan predikatskom izrazu NOT IN.

2.29. Prikazati imena, prezimena i prosečne ocene studenata, čija je prosečna ocena veća od ukupne prosečne ocene svih studenata.

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

```
SELECT student.IME, student.PREZIME, AVG(OCENA)
FROM student, polaže
WHERE student.BR_IND = polaže.BR_IND
GROUP BY polaže.BR_IND, student.IME, student.PREZIME
      AND AVG(OCENA) > SELECT(AVG(OCENA)
                             FROM polaže)
```

Zavisni ugnježdeni upiti

2.30. Prikazati brojeve indeksa, imena studenata, nazive predmete koje su položili i to sa ocenom koja je veća od prosečne ocene na tom predmetu.

```
SELECT BR_IND, IME, PREZIME, NAZIV:PRED, OCENA
FROM student, polaže pol_1, predmet
WHERE student.BR_IND = pol_1.BR_IND
      AND pol_1.SIF_PRED = predmet.SIF_PRED
      AND pol_1.OCENA > ANY (SELECT AVG(OCENA)
                             FROM polaže pol_2
                             WHERE pol_2.SIF_PRED = pol_1.SIF_PRED)
```

U ovom slučaju podupit zavisi od podataka koji se selektuju u glavnom upitu. Selektovanje torki podupita se vrši za svaku torku glavnog upita po jednom. □

Uniranje podataka

2.31. Prikazati imena, prezimena, zvanja i plate svih nastavnika koji zarađuju manje od prosečne plate za to zvanje ili nemaju svog rukovodioca.

```
SELECT nastavnik.IME_NAS, nastavnik.PREZ_NAS, nastavnik.ZVANJE,
nastavnik.PLT
FROM nastavnik n1
WHERE PLT < (SELECT AVG(PLT)
              FROM nastavnik n2
              WHERE n1.ZVANJE = n2.ZVANJE
              GROUP BY n2.zvanje)

UNION
SELECT nastavnik.IME_NAS, nastavnik.PREZ_NAS, nastavnik.ZVANJE,
nastavnik.PLT
FROM nastavnik
WHERE SIF_RUKOV IS NULL.
```

Kreiranje pogleda pueom SQL-a

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

2.32. Kreirati pogled kojim se prikazuju nazivi predmeta koje je položio bar jedan student.

```
CREATE VIEW položeni_predmeti(NAZIV_PRED) AS
SELECT NAZIV_PRED
FROM predmet
WHERE predmet.SIF_PRED IN (SELECT DISTINCT SIF
```

Ažuriranje baze podataka upotrebom jezika SQL

2.33. Ubaciti u tabelu *nastavnik* podatke o nastavniku Mirku Iliću, šifra 101, koji je rođen 15.01.1956, iz Novog Sada, u zvanju docenta, radi na katedri 2 i nema neposrednog rukovodioca.

```
INSERT INTO nastavnik
VALUES(101, "Mirko", "Ilić", "15.01.1956", 021, "docent", 2, 0, NULL)
```

Specijalna konstanta NULL označava da obeležje SIF_RUKOV dobija nula-vrednost.

Brisanje postojećih torki

2.34. Izbrisati iz relacije *predaje* sve predmete sa šifrom 20.

```
DELETE FROM predaje
WHERE SIF_PRED = 20.
```

Modifikacija postojećih torki

2.35. Svim nastavnicima u zvanju docenta ili vanrednog profesora povećati platu za 10%.

```
UPDATE nastavnik
SET PLT = PLT * 1.10
WHERE ZVANJE IN ('docent' OR 'vanredni profesor')
```

Definisanje fizičke strukture baze podataka putem jezika SQL

2.36. Kreirati indekse nad primarnim ključevima za relacije *student*, *grad*, *predmet* i *polaže*.

```
CREATE UNIQUE INDEX idx_student
ON student (BR_IND)
```

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

```
CREATE UNIQUE INDEX idx_grad  
ON grad (PTT ASC)
```

```
CREATE UNIQUE INDEX idx_predmet  
ON predmet (SIF_PRED)
```

```
CREATE UNIQUE INDEX idx_polaže  
ON polaže (BR_IND, SIF_PRED)
```

Primer 3.

Data je sledeća šema baze podataka $S = (S, I)$, pri čemu je skup šema relacija:

$S = \{ \text{Dobavljač}(\{ID_DOBAVLJAČA, NAZIV, STATUS, GRAD\}, \{ID_DOBAVLJAČA\}),$
 $\text{Deo}(\{ID_DETALJA, NAZIV, BOJA, TEŽINA, GRAD\}, \{ID_DETALJA\}),$
 $\text{Proizvod}(\{ID_PROIZVODA, NAZIV, GRAD\}, \{ID_PROIZVODA\}),$
 $\text{SPJ}(\{ID_DOBAVLJAČA, ID_DETALJA, ID_PROIZVODA, KOLIČINA\},$
 $\{ID_DOBAVLJAČA, ID_DETALJA, ID_PROIZVODA\}) \quad \},$

a skup međurelacionih ograničenja:

$I = \{ \text{SPJ}[ID_DOBAVLJAČA] \subseteq \text{Dobavljač}[ID_DOBAVLJAČA],$
 $\text{SPJ}[ID_DETALJA] \subseteq \text{Deo}[ID_DETALJA],$
 $\text{SPJ}[ID_PROIZVODA] \subseteq \text{Proizvod}[ID_PROIZVODA] \quad \}.$

Obeležja šema relacija imaju sledeća značenja:

GRAD – grad u kome je lociran dobavljač (skladište delova proizvoda ili lokacija gde se proizvod proizvodi),

STATUS – status dobavljača (redovan, povremen, vanredan a u relaciji dobavljač izražava se numerički npr. 10, 20, 30)

KOLIČINA – količina detalja koju dati dobavljač isporučuje za dati proizvod.

Neka je $rbp = \{ \text{dobavljač}, \text{deo}, \text{proizvod}, \text{spj} \}$ baza podataka nad šemom S .

Putem naredbi SQL-a realizovati sledeće upite (bez zalaženja u sintaksne formalizme konkretnih softvera za rukovanje bazama podataka (PROGRESS, ORACLE, FoxPro)).

PROSTI UPITI

3.1. Prikazati *sve* podatke o *svim* proizvodima.

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

```
SELECT ID_PROIZVODA, NAZIV, GRAD  
FROM proizvod
```

ili:

```
SELECT *  
FROM proizvod
```

3.2. Prikazati podatke o svim proizvodima, koji se proizvode u Zrenjaninu.

```
SELECT ID_PROIZVODA, NAZIV, GRAD  
FROM proizvod  
WHERE GRAD= "Zrenjanin"
```

3.3. Prikazati spisak šifara dobavljača uređen po šiframa dobavljača, koji isporučuju detalje za proizvod sa šifrom P1.

```
SELECT DISTINCT ID_DOBAVLJAČA  
FROM spj  
WHERE ID_PROIZVODA ="P1"  
ORDER BY ID_DOBAVLJAČA
```

3.4. Prikazati spisak svih isporuka, u kojima se količina detalja nalazi u opsegu 300 do 750 komada uključujući i te vrednosti.

```
SELECT ID_DOBAVLJAČA, ID_DETALJA, ID_PROIZVODA, KOLIČINA  
FROM spj  
WHERE KOLIČINA >= 300 AND KOLIČINA<=750  
(ILI BETWEEN 300 AND 750)
```

3.5. Prikazati spisak svih kombinacija "boja detalja - grad", gde se skladišti detalj, isključujući iste vrednosti parova (boja - grad).

```
SELECT DISTINCT BOJA, GRAD  
FROM deo
```

3.6. Prikazati spisak svih isporuka, u kojima količina nije neodređena.

```
SELECT ID_DOBAVLJAČA, ID_DETALJA, ID_PROIZVODA, KOLIČINA  
FROM spj  
WHERE KOLIČINA IS NOT NULL  
(ILI WHERE KOLIČINA=KOLIČINA)
```

3.7. Prikazati šifre proizvoda i imena gradova, u kojima se proizvodi proizvode, i to one gradove čije je drugo slovo imena "O".

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

```
SELECT ID_PROIZVODA, GRAD
FROM proizvod
WHERE GRAD LIKE '__O%'
```

POVEZIVANJE TABELA (SPOJ TABELA)

3.8. Prikazati sve trojke “šifra dobavljača, broj detalja i broj proizvoda”, takve, da su njihovi dobavljač, detalj i proizvod iz istih gradova.

```
SELECT ID_DOBAVLJAČA, ID_DETALJA, ID_PROIZVODA
FROM dobavljač d, deo, proizvod p
WHERE d.GRAD = deo.GRAD
AND deo.GRAD = p.GRAD
```

3.9. Prikazati sve trojke “šifra dobavljača, šifra detalja i šifra proizvoda”, takve, da su dobavljači, detalji i proizvodi date trojke iz različitih gradova.

```
SELECT ID_DOBAVLJAČA, ID_DETALJA, ID_PROIZVODA
FROM dobavljač d, deo, proizvod p
WHERE NOT (d.GRAD = deo.GRAD AND deo.GRAD = p.GRAD)
```

3.10. Prikazati šifre detalja, koje isporučuje bilo koji dobavljač iz Novog Sada, za proizvode, koji se, takođe, proizvode u Novom Sadu.

```
SELECT DISTINCT ID_DETALJA
FROM spj, dobavljač d, proizvod p
WHERE spj.ID_DOBAVLJAČA = d.ID_DOBAVLJAČA
AND spj.ID_PROIZVODA = p.ID_PROIZVODA
AND d.GRAD = "Novi Sad"
AND p.GRAD = "Novi Sad"
```

3.11. Prikazati šifre detalja, koje isporučuje bilo koji dobavljač iz Zrenjanina.

```
SELECT DISTINCT ID_DETALJA
FROM spj, dobavljač d
WHERE spj.ID_DOBAVLJAČA = d.ID_DOBAVLJAČA
AND d.GRAD = "Zrenjanin"
```

3.12. Prikazati šifre detalja, koji se dobavljaju zbog bilo kog proizvoda, koga isporučuje dobavljač, koji je lociran u istom gradu u kome se proizvodi taj proizvod.

```
SELECT DISTINCT ID_DETALJA
FROM spj, dobavljač d, proizvod p
WHERE spj.ID_DOBAVLJAČA = d.ID_DOBAVLJAČA
AND spj.ID_PROIZVODA = p.ID_PROIZVODA
AND d.GRAD = p.GRAD
```

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

3.13. Prikazati šifre proizvoda, za koje se detalji dobavljaju bar od jednog dobavljača, ali da dobavljač nije iz istog grada kao i proizvod.

```
SELECT DISTINCT ID_PROIZVODA
FROM spj, dobavljač d, proizvod p
WHERE spj.ID_DOBAVLJAČA = d.ID_DOBAVLJAČA
AND spj.ID_PROIZVODA = p.ID_PROIZVODA
AND d.GRAD <> p.GRAD
```

(Napomena: simboli <> ili $\neg$ = ili != su simboli za operaciju *različito* u zavisnosti od konkretnog softvera).

MANIPULACIJA PODACIMA

Klauzula EXISTS

3.14. Korektni upit:

```
SELECT S.ID_DOBAVLJAČA
FROM dobavljač d
WHERE d.GRAD != ANY (SELECT deo.GRAD
                     FROM deo)
```

uopšte ne ostvaruje pregled šifara dobavljača, koji se nalaze u gradovima, koji se ne podudaraju sa gradovima, gde se nalaze detalji. Umesto toga, on daje pregled broja dobavljača, takvih da se grad u kome su oni nalaze, ne podudara sa nijednim gradom, gde se skladište detalji. Ekvivalentni zapis uz pomoć **EXISTS** daje jasniju korektniju interpretaciju:

```
SELECT d.ID_DOBAVLJAČA
FROM dobavljač d
WHERE EXISTS (SELECT deo.GRAD
              FROM deo
              WHERE deo.GRAD != d.GRAD)
```

(“Prikazati šifre takvih dobavljača, da postoji neki grad skladištenja detalja, koji se razlikuje od grada, gde se nalazi dati dobavljač”). Prirodna intuitivna interpretacija !=ANY kao “ne podudara se sa nekima” je nekorektna i veoma dvosmislena.

PODUPITI

3.15. Prikazati nazive dobavljača, koji snabdevaju detaljem D2.

```
SELECT NAZIV
FROM dobavljač
WHERE ID_DOBAVLJAČA IN
```


SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

```
(SELECT ID_DOBAVLJAČA  
FROM spj  
WHERE ID_DETALJA = "D2")
```

ili:

```
SELECT d.NAZIV  
FROM dobavljač d, spj  
WHERE d.ID_DOBAVLJAČA = spj.ID_DOBAVLJAČA  
AND spj.ID_DETALJA = "D2"
```

PODUPIT SA NEKOLIKO NIVOA SLOŽENOSTI

3.16. Prikazati nazive dobavljača, koji isporučuju bar jedan deo crvene boje.

```
SELECT NAZIV  
FROM dobavljač  
WHERE ID_DOBAVLJAČA IN  
      (SELECT ID_DOBAVLJAČA  
       FROM spj  
       WHERE ID_DETALJA IN  
             (SELECT ID_DETALJA  
              FROM deo  
              WHERE BOJA = "crvena"))
```

KORELISANI PODUPIT

3.17. Prikazati nazive dobavljača, koji isporučuju detalj D2.

```
SELECT NAZIV  
FROM dobavljač  
WHERE "D2" IN  
      (SELECT ID_DETALJA  
       FROM spj  
       WHERE spj.ID_DOBAVLJAČA = d.ID_DOBAVLJAČA)
```

Objašnjenje. Ovaj primer se razlikuje od prethodnih po tome, što se unutrašnji podupit ne izvršava pre spoljašnjeg, zato što taj unutrašnji upit zavisi od promenljive d.ID_DOBAVLJAČA, čije se značenje menja u odnosu na to kako sistem proverava različite redove tabele *dobavljač*. Upit se izračunava na sledeći način:

a) Sistem proverava prvi red tabele *dobavljač*. Na primer, neka je to DOB1. Tako imamo:

```
(SELECT ID_DETALJA  
FROM spj  
WHERE ID_DOBAVLJAČA = "DOB1"),
```

a kao rezultat se dobija ('D1', 'D2', 'D3', 'D4', 'D5', 'D6'). Sada se može izvršiti obrada za DOBL. Ime je KOMERC, i ono se dobija tada i samo tada, kada 'D2' pripada tom skupu, što je istina.

b) Dalje sistem radi za sledećeg dobavljača itd.

Ovaj upit se zove **KORELISANI**. To je takav podupit, čiji rezultat zavisi od neke promenljive. Ta promenljiva dobija svoju vrednost u nekom spoljašnjem upitu. Obrada takvog podupita se ponavlja za svaku vrednost promenljive u upitu, a ne izvršava se svaki put.

Ili:

```
SELECT NAZIV
FROM dobavljač dX
WHERE "D2" IN
      (SELECT ID_DETALJA
       FROM spj
       WHERE ID_DOBAVLJAČA = dX.ID_DOBAVLJAČA)
```

Za svaku moguću vrednost dX izvršava se sledeće:

- izračunati podupit i dobiti skup šifara detalja npr. D;
- izdvojiti, u odnosu na rezultujući skup, vrednost dX.NAZIV, ako i samo ako D2 pripada skupu D.

Simbol *d* označava i baznu tabelu, a takođe, i promenljivu, koja se odnosi na skup zapisa te bazne tabele.

Uvođenje sinonima nikada nije greška, a često je i neophodnost.

SLUČAJ UPOTREBE JEDNE ISTE TABELE U PODUPITU I U SPOLJAŠNJEM UPITU

3.18. Prikazati brojeve dobavljača, koji isporučuju bar jedan detalj, koje isporučuje dobavljač D2.

```
SELECT DISTINCT ID_DOBAVLJAČA
FROM spj
WHERE ID_DETALJA IN
      (SELECT ID_DETALJA
       FROM spj
       WHERE ID_DOBAVLJAČA = "D2")
```

Primerimo da poziv na relaciju *spj* u podupitu ne označava isto kao poziv na relaciju *spj* u spoljašnjem upitu. U stvarnosti, dva imena *spj* označavaju *različite promenljive*.

Koristićemo sinonime:

```
SELECT DISTINCT spjX.ID_DOBAVLJAČA
```

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

```
FROM spj spjX
WHERE spjX.ID_DETALJA IN
      (SELECT spjY.ID_DETALJA
       FROM spj spjY
       WHERE spjY.ID_DOBAVLJAČA = "D2")
```

Ekvivalentan upit sa upotrebom sjedinjenja je:

```
SELECT DISTINCT spjX.ID_DOBAVLJAČA
FROM spj spjX, spj spjY
WHERE spjX.ID_DETALJA = spjY.ID_DETALJA
AND spjY.ID_DOBAVLJAČA = "D2"
```

SLUČAJ KADA SE I U KORELISANOM I U SPOLJAŠNJEM UPITU KORISTI ISTA TABELA

3.19. Prikazati brojeve svih detalja, koje isporučuju više od jednog dobavljača.

```
SELECT DISTINCT spjX.ID_DETALJA
FROM spj spjX
WHERE spjX.ID_DETALJA IN
      (SELECT spjY.ID_DETALJA
       FROM spj spjY
       WHERE spjY.ID_DOBAVLJAČA != spjX.ID_DOBAVLJAČA)
```

Ovo se može i ovako opisati:

“Redom za svaki red tabele *spj*, nazovimo je *spjX*, dati vrednosti ID_DETALJA, ako i samo ako se ta vrednost nalazi u nekom redu, nazovimo ga *spjY*, tabele *spj*, a vrednost ID_DOBAVLJAČA tog reda, se ne nalazi u vrednosti reda *spjX*.”

PODUPIT SA OPERATOROM IZJEDNAČAVANJA RAZLIČITIM OD IN

3.20. Prikazati šifre dobavljača, koji se nalaze u istom gradu, kao i dobavljač D1.

```
SELECT ID_DOBAVLJAČA
FROM dobavljač
WHERE GRAD =
      (SELECT GRAD
       FROM dobavljač
       WHERE ID_DOBAVLJAČA = "D1")
```

Nekada korisnik može da zna unapred da će rezultat unutrašnjeg podupita biti samo jedna vrednost, kao u ovom primeru. U tom slučaju se može koristiti umesto operatora IN jednostavniji operator upoređivanja (npr. =, > itd.). Ipak, ako podupit vraća više od jedne vrednosti a ne koristi se operator IN nastaje greška. Greška neće nastati ako unutrašnji podupit ne vrati nijednu vrednost.

EGZISTENCIJALNI KVANTIFIKATOR

3.21. Prikazati nazive dobavljača, koji isporučuju detalj DEO2.

```
SELECT NAZIV
FROM dobavljač d
WHERE EXISTS
    (SELECT *
     FROM spj
     WHERE ID_DOBAVLJAČA = d.ID_DOBAVLJAČA
     AND ID_DETALJA = "DEO2")
```

Objašnjenje. EXISTS (postoji) predstavlja *kvantifikator postojanja* - u formalnoj logici. Neka simbol “x” označava neku proizvoljnu promenljivu. Tada u logici *predikat sa egzistencijalnim kvantifikatorom* EXISTS x (predikat - koji zavisi - od - x) dobija značenje *istina* tada i samo tada, kada “predikat koji zavisi od x” ima značenje *istina* u bilo kojem značenju promenljive x.

U jeziku SQL predikat sa egzistencijalnim kvantifikatorom ima oblik:

EXISTS (SELECT * FROM . . .).

Takav izraz se smatra istinitim samo tada kada je rezultat podupita, koji se predstavlja u obliku “SELECT * FROM . . .”, neprazan skup, drugim rečima, samo tada, kada postoji neki slog u tabeli u frazi FROM podupita, koji zadovoljava uslov WHERE toga podupita. (U praksi će taj podupit uvek biti korelisani skup).

U našem primeru se za svaku vrednost kolone NAZIV proverava da li je istinit uslov postojanja. Na primer, prva vrednost polja NAZIV je “KOMERC”. Tada je odgovarajuća vrednost ID_DOBAVLJAČA - D1. Da li je prazan skup slogova iz relacije *spj*, koji sadrže ID_DOBAVLJAČA koji imaju vrednost D1, i ID_DETALJA koji je jednak sa DEO2? Ako je odgovor odričan, onda postoji slog (n-torka) u relaciji (tabeli) *spj* sa vrednošću obeležja ID_DOBAVLJAČA, koja je jednaka D1, i brojem detalja, koji je jednak sa DEO2, i, sledi da je “KOMERC” vrednost rezultata.

EXISTS predstavlja jednu od najvažnijih mogućnosti jezika SQL. Faktički se svaki upit, koji može biti izražen putem IN, može realizovati pomoću EXISTS. Ali, obrnuto ne važi.

UPIT KOJI KORISTI NOT EXISTS

3.22. Prikazati nazive dobavljača, koji ne isporučuju detalj DEO2.

```
SELECT NAZIV
FROM dobavljač d
WHERE NOT EXISTS
    (SELECT *
     FROM spj
```

```
WHERE ID_DOBAVLJAČA = d.ID_DOBAVLJAČA  
AND ID_DETALJA = "DEO2")
```

Ovaj upit je moguće izraziti rečima: “Prikazati imena dobavljača, za koje ne postoji isporuka, koja ih povezuje sa detaljem DEO2.

Između ostalog, podupit koji sledi iza EXISTS oblika “SELECT *”, može imati oblik “SELECT *ime-polja* FROM ...”. Ionako ono u praksi gotovo uvek ima oblik “SELECT *”.

UPIT KOJI KORISTI *NOT EXISTS*

3.23. Prikazati nazive dobavljača, koji isporučuju sve detalje.

U principu, FORALL je to što je nužno u ovom upitu. Mi želimo da kažemo sledeće: “Prikazati nazive dobavljača, onih, da ZA SVE (FORALL) detalje POSTOJI (EXISTS) slog tabele *spj*, koji govori o tome da dati dobavljač isporučuje dati detalj”. Nažalost, u SQL-u ne postoji kvantifikator FORALL. Ali može:

FORALL x (p) = NOT (EXISTS x (NOT (p))).

Zato možemo upit “dobavljači takvi, da ZA SVE detalje POSTOJI slog tabele *spj*, koji govori o tome da dati dobavljač isporučuje taj detalj” ekvivalentno izraziti “dobavljači takvi, da NE POSTOJE detalji, takvi da NE POSTOJI slog tabele *spj*, koji govori o tome da dati dobavljač isporučuje taj detalj”.

```
SELECT NAZIV  
FROM dobavljač d  
WHERE NOT EXISTS  
      (SELECT *  
        FROM deo  
        WHERE NOT EXISTS  
              (SELECT *  
                FROM spj  
                WHERE ID_DOBAVLJAČA = d.ID_DOBAVLJAČA  
                  AND ID_DETALJA = deo.ID_DETALJA))
```

Dati izraz može se izraziti rečima: “Prikazati nazive dobavljača takvih, da ne postoje detalji, koje oni ne isporučuju”.

UPIT KOJI KORISTI *NOT EXISTS*

3.24. Prikazati brojeve dobavljača, koji isporučuju bar one detalje, koje isporučuje dobavljač D2.

Možemo razbiti ovaj upit na više upita. Tako dobijamo:

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

```
SELECT ID_DETALJA
```

```
FROM spj
```

```
WHERE ID_DOBAVLJAČA = "D2"
```

Možemo koristiti tabelu PRIVREMENA u koju smestimo ove rezultate a zatim je koristimo na sledeći način:

```
SELECT DISTINCT ID_DOBAVLJAČA
```

```
FROM spj spjX
```

```
WHERE NOT EXISTS
```

```
    (SELECT *
```

```
      FROM PRIVREMENA
```

```
      WHERE NOT EXISTS
```

```
        (SELECT *
```

```
          FROM spj spjY
```

```
          WHERE spjY.ID_DOBAVLJAČA = spjX.ID_DOBAVLJAČA
```

```
          AND spjY.ID_DETALJA = PRIVREMENA.ID_DETALJA))
```

A ovaj upit izgleda i ovako (ako se ne koristi tabela PRIVREMENA):

```
SELECT DISTINCT ID_DOBAVLJAČA
```

```
FROM spj spjX
```

```
WHERE NOT EXISTS
```

```
    (SELECT *
```

```
      FROM spj spjY
```

```
      WHERE ID_DOBAVLJAČA = "D2"
```

```
      AND NOT EXISTS
```

```
        (SELECT *
```

```
          FROM spj spjZ
```

```
          WHERE spjZ.ID_DOBAVLJAČA = spjX.ID_DOBAVLJAČA
```

```
          AND spjZ.ID_DETALJA = spjY.ID_DETALJA))
```

FUNKCIJA U KORELISANOM PODUPITU

3.25. Prikazati broj dobavljača, sastav i grad za sve one dobavljače, kod kojih je status veći ili jednak srednjem statusu za konkretni grad.

```
SELECT ID_DOBAVLJAČA, STATUS, GRAD
```

```
FROM dobavljač dX
```

```
WHERE STATUS >=
```

```
    (SELECT AVG(STATUS)
```

```
      FROM dobavljač dY
```

```
      WHERE dY.GRAD = dX.GRAD)
```

Uključiti srednji status za svaki grad je nemoguće iz sledećih razloga:

Svaki izraz u frazi SELECT mora imati *jedinstveno značenje za celu grupu*, tj. ono može biti ili samo polje, koje se nalazi u GROUP BY klauzuli, ili aritmetički izraz koji

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

uključuje to polje, ili konstanta, ili takva funkcija kao što je SUM, koja operiše nad svim vrednostima datog polja u grupi.

Ako polje (odnosno, obeležje), po kome je izvršeno grupisanje, ima neke nedefinisane vrednosti, za svaku od njih se formira posebna grupa.

KLAUZULA *HAVING*

Izraz u frazi *HAVING* mora imati jedinstvenu vrednost za grupu.

3.26. Prikazati šifre detalja za sve detalje, koje isporučuje više od jednog dobavljača.

```
SELECT DISTINCT ID_DETALJA
FROM spj spjX
WHERE 1<
    (SELECT COUNT(*)
     FROM spj spjY
     WHERE spjY.ID_DETALJA = spjX.ID_DETALJA))
```

3.27. Sledeći upit u kome se umesto relacije *spj* koristi relacija *deo* je jasniji:

```
SELECT ID_DETALJA
FROM deo
WHERE 1<
    (SELECT COUNT(ID_DETALJA)
     FROM spj
     WHERE ID_DETALJA = deo.ID_DETALJA))
```

A evo upita uz pomoć klauzule *EXISTS*:

```
SELECT ID_DETALJA
FROM deo
WHERE EXISTS
    (SELECT *
     FROM spj spjX
     WHERE spjX.ID_DETALJA = deo.ID_DETALJA
     AND EXISTS
        (SELECT *
         FROM spj spjY
         WHERE spjY.ID_DETALJA = deo.ID_DETALJA
         AND spjY.ID_DOBAVLJAČA != spjX.ID_DOBAVLJAČA))
```

Postoje neki zadaci takvog karaktera, za koje su *GROUP BY* i *HAVING* neadekvatni.

Konstrukcija GROUP BY radi samo na jednom nivou. Nije moguće razbiti svaku od tih grupa na grupe nižeg nivoa, a zatim primeniti neku standardnu funkciju, na primer, SUM ili AVG na svakom nivou grupisanja.

SVEOBUHVAJNI PRIMER

3.28. Prikazati brojeve detalja, težinu u gramima, boju i maksimalni obim isporuke za sve crvene i plave delove, takve, da je opšti obim njihove isporuke veći od 350, isključujući pri tome iz opšteg proseka, takve isporuke, za koje je količina manja ili jednaka 200 detalja. Rezultat urediti po opadajućem redosledu brojeva detalja, a u rastućoj vrednosti maksimalnog obima isporuke.

```
SELECT deo.ID_DETALJA, "težina u gramima = ", deo.TEŽINA, deo.BOJA,
      "maksimalni obim isporuke = ", MAX(spj.KOLIČINA)
FROM deo, spj
WHERE deo.ID_DETALJA = spj.ID_DETALJA
AND deo.BOJA IN ("crvena", "plava")
AND spj.KOLIČINA > 200
GROUP BY deo.ID_DETALJA, deo.TEŽINA, deo.BOJA
HAVING SUM(KOLIČINA)>350
ORDER BY 6, deo.ID_DETALJA DESC
```

1. FROM. U frazi FROM kreira se nova tabela, koja je dekartov proizvod tabela *deo* i *spj*.
2. WHERE. Isključuju se zapisi koji ne zadovoljavaju uslov.
3. GROUP BY. Rezultat koraka 2 se grupiše po vrednostima polja u GROUP BY. *Napomena.* Teorijski bi bilo dovoljno u GROUP BY koristiti samo deo.ID_DETALJA, zbog toga što su deo.TEŽINA i deo.BOJA jednoznačno određeni brojem detalja. Ipak, ako deo.TEŽINA i deo.BOJA budu izostavljeni iz fraze GROUP BY, desiće se greške, ako su oni uključeni u frazu SELECT. ***Osnovni problem se sastoji u tome, što sistem DB2 ne podržava primarne ključeve.***
4. HAVING. Isključuju se grupe koje ne zadovoljavaju uslov.
5. SELECT. Svaka grupa, koja se dobija u koraku 4, generiše jedinstveni red za rezultat. Prvo, iz grupe se izdvajaju brojevi detalja, težina, boja i maksimalni obim isporuke. Zatim, postavljaju se konstante "težina u gramima = " itd.
6. ORDER BY. Rezultat koraka 5 se uređuje u odnosu na frazu u ORDER BY i to u odnosu na atribut koji je 6-ti po redu u redosledu navođenja atributa iza SELECT izraza (a to je atribut MAX(spj.KOLIČINA)).

V E Ź B E

Kao i u prethodnom primeru, svi upiti se odnose na istu bazu podataka.

$S = \{ \text{Dobavljač}(\{ID_DOBAVLJAČA, NAZIV, STATUS, GRAD\}, \{S_DOBAVLJAČA\}),$
 $\text{Deo}(\{ID_DETALJA, NAZIV, BOJA, TEŽINA, GRAD\}, \{ID_DETALJA\}),$
 $\text{Proizvod}(\{ID_PROIZVODA, NAZIV, GRAD\}, \{ID_PROIZVODA\}),$
 $\text{SPJ}(\{ID_DOBAVLJAČA, ID_DETALJA, ID_PROIZVODA, KOLIČINA\},$

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

{ID_DOBAVLJAČA, ID_DETALJA, ID_PROIZVODA}}),

U ovom delu upiti su poredani po složenosti. Upiti 3.40-3.46 su veoma složeni.

PODUPITI

3.29. Prikazati nazive proizvoda, za koje detalje isporučuje dobavljač D1.

```
SELECT NAZIV
FROM dobavljač
WHERE ID_DETALJA IN
      (SELECT ID_DETALJA
       FROM spj
       WHERE ID_DOBAVLJAČA = "D1")
```

3.30. Prikazati nazive detalja, koje isporučuje dobavljač D1.

```
SELECT DISTINCT NAZIV
FROM deo
WHERE ID_DETALJA IN
      (SELECT ID_DETALJA
       FROM spj
       WHERE ID_DOBAVLJAČA = "D1")
```

3.31. Prikazati šifre detalja, koji se dobavljaju zbog *bar jednog* proizvoda koji se proizvode u Novom Sadu..

```
SELECT DISTINCT ID_DETALJA
FROM spj
WHERE ID_PROIZVODA IN
      (SELECT ID_PROIZVODA
       FROM proizvod
       WHERE GRAD = "Novi Sad")
```

3.32. Prikazati šifre proizvoda, koji koriste barem jedan detalj, koga isporučuje dobavljač D1.

```
SELECT ID_PROIZVODA
FROM spj
WHERE ID_DETALJA IN
      (SELECT ID_DETALJA
       FROM spj
       WHERE ID_DOBAVLJAČA = "D1")
```

3.33. Prikazati šifre dobavljača, koji snabdevaju bar jednim detaljem koga isporučuje dobavljač, koji isporučuje bar jedan deo crvene boje.

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

```
SELECT DISTINCT ID_DOBAVLJAČA
FROM spj
WHERE ID_DETALJA IN
    (SELECT ID_DETALJA
     FROM spj
     WHERE ID_DOBAVLJAČA IN
         (SELECT ID_DOBAVLJAČA
          FROM spj
          WHERE ID_DETALJA IN
              (SELECT ID_DETALJA
               FROM deo
               WHERE BOJA = "crvena"))))
```

3.34. Prikazati šifre dobavljača, koji imaju status manji nego dobavljač D1.

```
SELECT DISTINCT ID_DOBAVLJAČA
FROM dobavljač
WHERE STATUS <
    (SELECT STATUS
     FROM dobavljač
     WHERE ID_DOBAVLJAČA = "D1")
```

3.35. Prikazati brojeve dobavljača, koji isporučuju detalje za bilo koji proizvod sa detaljem DEO1 u količini, većoj od srednjeg obima isporuke detalja DEO1 za taj proizvod. *Primedba.* Neophodno je koristiti funkciju AVG.

```
SELECT ID_DOBAVLJAČA
FROM spj spjX
WHERE ID_DETALJA = "DEO1"
    AND KOLIČINA >
        (SELECT AVG(KOLIČINA)
         FROM spj spjY
         WHERE ID_DETALJA = "DEO1"
         AND spjY.ID_PROIZVODA = spjX.ID_PROIZVODA)
```

KVANTIFIKATOR *EXISTS*

3.36. Ponovite upit 3.31. pomoću izraza EXISTS.

(3.31.. Prikazati šifre detalja, koji se dobavljaju zbog *bar jednog* proizvoda koji se proizvode u Novom Sadu.)

```
SELECT DISTINCT ID_DETALJA
FROM spj
WHERE EXISTS
```

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

```
(SELECT *  
FROM deo  
WHERE ID_PROIZVODA = spj.ID_PROIZVODA  
AND GRAD = "Novi Sad")
```

3.37. Ponovite upit 3.32. pomoću izraza EXISTS.

(3.32. Prikazati šifre proizvoda, koji koriste barem jedan detalj, koga isporučuje dobavljač D1.)

```
SELECT DISTINCT spjX.ID_PROIZVODA  
FROM spj spjX  
WHERE EXISTS  
  (SELECT *  
   FROM spj spjY  
   WHERE spjY.ID_DETALJA = spjX.ID_DETALJA  
   AND spjY.ID_DOBAVLJAČA = "D1")
```

3.38. Prikazati šifre proizvoda, za koje nijedan dobavljač iz Subotice ne isporučuje deo crvene boje.

```
SELECT ID_PROIZVODA  
FROM deo  
WHERE NOT EXISTS  
  (SELECT *  
   FROM spj  
   WHERE ID_PROIZVODA = deo.ID_PROIZVODA  
   AND ID_DETALJA IN  
     (SELECT ID_DETALJA  
      FROM deo  
      WHERE BOJA = "crvena")  
   AND ID_DOBAVLJAČA IN  
     (SELECT ID_DOBAVLJAČA  
      FROM dobavljač  
      WHERE GRAD = "Subotica"))
```

3.39. Prikazati šifre proizvoda, čije delove u potpunosti isporučuje dobavljač D1.

```
SELECT DISTINCT ID_PROIZVODA  
FROM spj spjX  
WHERE NOT EXISTS  
  (SELECT *  
   FROM spj spjY  
   WHERE spjY.ID_PROIZVODA = spjX.ID_PROIZVODA  
   AND spjY.ID_DOBAVLJAČA != "D1")
```

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

3.40. Prikazati šifre detalja, koji se isporučuju za *sve* proizvode koji se proizvode u Zrenjaninu.

```
SELECT DISTINCT ID_DETALJA
FROM spj spjX
WHERE NOT EXISTS
    (SELECT *
     FROM proizvod
     WHERE GRAD = "Zrenjanin"
     AND NOT EXISTS
        (SELECT *
         FROM spj spjY
         WHERE spjY.ID_DETALJA = spjX.ID_DETALJA
         AND spjY.ID_PROIZVODA = deo.ID_PROIZVODA))
```

3.41. Prikazati šifre dobavljača, koji isporučuju jedan isti detalj za sve proizvode.

```
SELECT DISTINCT ID_DOBAVLJAČA
FROM spj spjX
WHERE EXISTS
    (SELECT ID_DETALJA
     FROM deo
     WHERE NOT EXISTS
        (SELECT ID_PROIZVODA
         FROM proizvod
         WHERE NOT EXISTS
            (SELECT *
             FROM spj spjZ
             WHERE spjZ.ID_DOBAVLJAČA = spjX.ID_DOBAVLJAČA
             AND spjZ.ID_DETALJA = deo.ID_DETALJA
             AND spjZ.ID_PROIZVODA = proizvod.ID_PROIZVODA))))
```

Ovo je veoma složen upit i može se parafrazirati:

“Prikazati dobavljače (spjX.ID_DOBAVLJAČA), i to one za koje važi, da postoji detalj (deo.ID_DETALJA), takav, da ne postoji nijedan proizvod (proizvod.ID_PROIZVODA), za koga važi da dati dobavljač ne isporučuje dati detalj za taj proizvod.”

Drugim rečima, potrebno je prikazati takve dobavljače, tako da postoji detalj, koga oni isporučuju za sve proizvode.

Primetimo, da se fraze “SELECT ID_DETALJA FROM...” i “SELECT ID_PROIZVODA FROM...” koriste dva puta kao oblast važenja kvantifikatora EXISTS. Pri tome, upotreba “SELECT *” ne bi bila nekorektna, ali “SELECT ID_DETALJA”, na primer, je bliže intuitivnoj formalizaciji - potrebno je da postoji *neki detalj*, koji se identifikuje brojem detalja, a ne jednostavno zapis u tabeli o detaljima.

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

3.42. Prikazati šifre proizvoda, za koje se isporučuju svi detalji, koje isporučuje dobavljač D1.

```
SELECT ID_PROIZVODA
FROM spj spjX
WHERE NOT EXISTS
    (SELECT ID_DETALJA
     FROM spj spjY
     WHERE spjY.ID_DOBAVLJAČA = "D1"
     AND NOT EXISTS
        (SELECT *
         FROM spj spjZ
         WHERE spjZ.ID_DETALJA = spjY.ID_DETALJA
         AND spjZ.ID_PROIZVODA = spjX.ID_PROIZVODA))
```

Za sledeća tri upita (3.43. – 3.45.) prevedite upite SQL-a u prirodni govor.

3.43.

```
SELECT DISTINCT ID_PROIZVODA
FROM spj spjX
WHERE NOT EXISTS
    (SELECT *
     FROM spj spjY
     WHERE spjY.ID_PROIZVODA = spjX.ID_PROIZVODA
     AND NOT EXISTS
        (SELECT *
         FROM spj spjZ
         WHERE spjZ.ID_DETALJA = spjY.ID_DETALJA
         AND spjZ.ID_DOBAVLJAČA = "D1"))
```

Prikazati brojeve proizvoda, koji koriste samo one detalje, koje isporučuje dobavljač D1.

3.44.

```
SELECT DISTINCT ID_PROIZVODA
FROM spj spjX
WHERE NOT EXISTS
    (SELECT *
     FROM spj spjY
     WHERE EXISTS
        (SELECT *
         FROM spj spjA
         WHERE spjA.ID_DOBAVLJAČA = "D1"
         AND spjA.ID_DETALJA = spjY.ID_DETALJA)
     AND NOT EXISTS
        (SELECT *
         FROM spj spjB
```

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

```
WHERE spjB.ID_DOBAVLJAČA = "D1"  
AND spjB.ID_DETALJA = spjY.ID_DETALJA  
AND spjB.ID_PROIZVODA = spjX.ID_PROIZVODA))
```

Prikazati brojeve proizvoda, za koje dobavljač D1 isporučuje nekoliko detalja, i to samo one koje on isporučuje.

3.45.

```
SELECT DISTINCT ID_PROIZVODA  
FROM spj spjX  
WHERE NOT EXISTS  
  (SELECT *  
   FROM spj spjY  
   WHERE EXISTS  
     (SELECT *  
      FROM spj spjA  
      WHERE spjA.ID_DOBAVLJAČA = spjY.ID_DOBAVLJAČA  
      AND spjA.ID_DETALJA IN  
        (SELECT ID_DETALJA  
         FROM deo  
         WHERE BOJA = "crvena")  
      AND NOT EXISTS  
        (SELECT *  
         FROM spj spjB  
         WHERE spjB.ID_DOBAVLJAČA = spjY.ID_DOBAVLJAČA  
         AND spjB.ID_PROIZVODA = spjX.ID_PROIZVODA)))
```

Prikazati brojeve proizvoda, za koje se isporučuju detalji od strane dobavljača, koji isporučuje bilo koji detalj crvene boje.

STANDARDNE FUNKCIJE

3.46. Prikazati ukupan broj proizvoda, za koje detalje isporučuje dobavljač D1.

```
SELECT COUNT(DISTINCT ID_PROIZVODA)  
FROM spj  
WHERE ID_DOBAVLJAČA = "D1"
```

3.47. Prikazati ukupnu količinu delova DEO1, koje isporučuje dobavljač D1.

```
SELECT SUM(KOLIČINA)  
FROM spj  
WHERE ID_DETALJA = "DEO1"  
  
AND ID_DOBAVLJAČA = "D1"
```

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

3.48. Za svaki isporučeni detalj za neki proizvod prikazati njegov broj, broj proizvoda i odgovarajuću ukupnu količinu detalja.

```
SELECT ID_DETALJA, ID_PROIZVODA, SUM(KOLIČINA)
FROM spj
GROUP BY ID_DETALJA, ID_PROIZVODA;
```

3.49. Prikazati brojeve proizvoda, za koje je prosečna vrednost isporuke detalja DEO1 veća nego najveći obim isporuke bilo kog detalja za proizvod P1.

```
SELECT ID_PROIZVODA
FROM spj
WHERE ID_DETALJA = "DEO1"
GROUP BY ID_PROIZVODA
HAVING AVG(KOLIČINA) >
    (SELECT MAX(KOLIČINA)
     FROM spj
     WHERE ID_PROIZVODA = "P1")
```

3.50. Prikazati brojeve dobavljača, koji isporučuju detalj DEO1 za bilo koji proizvod u količini, koja je veća od prosečnog obima isporuke detalja DEO1 za taj proizvod.

```
SELECT DISTINCT ID_DOBAVLJAČA
FROM spj spjX
WHERE ID_DETALJA = "DEO1"
AND KOLIČINA >
    (SELECT AVG(KOLIČINA)
     FROM spj spjY
     WHERE ID_DETALJA = "DEO1"
     AND spjY.ID_PROIZVODA = spjX.ID_PROIZVODA)
```

3.51. Napraviti redosledni spisak svih gradova, u kojima se nalazi bar jedan dobavljač, detalj ili proizvod.

```
SELECT GRAD FROM dobavljač
UNION
SELECT GRAD FROM deo
UNION
SELECT GRAD FROM proizvod
ORDER BY 1
```

MANIPULACIJA PODACIMA

IZMENA JEDNOG ZAPISA

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

3.52. Promeniti boju detalja DEO2 na žutu, povećati težinu za 5 i postaviti vrednost grada na "neizvesno" (NULL).

```
UPDATE deo
SET   BOJA = "žuta",
      TEŽINA = TEŽINA + 5,
      GRAD = NULL
WHERE ID_DETALJA = "DEO2"
```

3.53. Povećati status svih dobavljača iz Sombora na dvostruku vrednost.

```
UPDATE dobavljač
SET   STATUS = 2*STATUS
WHERE GRAD = "Sombor"
```

IZMENA SA PODUPITOM

3.54. Postaviti obim isporuke na nulu za sve dobavljače iz Sombora.

```
UPDATE spj
SET KOLIČINA = 0
WHERE "Sombor" =
      (SELECT GRAD
       FROM dobavljač d
       WHERE d.ID_DOBAVLJAČA = spj.ID_DOBAVLJAČA)
```

IZMENA NEKOLIKO TABELA

3.55. Promeniti šifru dobavljača sa D2 na D9.

```
UPDATE dobavljač
SET ID_DOBAVLJAČA = "D9"
WHERE ID_DOBAVLJAČA = "D2"
UPDATE spj
SET ID_DOBAVLJAČA = "D9"
WHERE ID_DOBAVLJAČA = "D2"
```

U frazi UPDATE se može navesti *samo jedna tabela*. Zato je izmenu potrebno izvršiti nad više tabela.

NAREDBA DELETE

3.56. Izbrisati podatke o dobavljaču D1.

```
DELETE
```


SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

```
FROM dobavljač  
WHERE ID_DOBAVLJAČA = "D1"
```

I opet, potrebno je ove izmene uraditi i u tabeli *spj* za dobavljača D1.

3.57. Izbrisati sve isporuke.

```
DELETE  
FROM spj
```

spj je još uvek poznata tabela, ali je sada prazna. Izbrisati sve zapise to nije isto kao i izbrisati celu tabelu (naredba DROP).

BRISANJE SA PODUPITOM

3.58. Izbrisati sve isporuke za dobavljače iz Zrenjanina.

```
DELETE  
FROM spj  
WHERE "Zrenjanin" =  
      (SELECT GRAD  
       FROM dobavljač d  
       WHERE d.ID_DOBAVLJAČA = spj.ID_DOBAVLJAČA)
```

NAREDBA INSERT

```
INSERT  
INTO tabela (polje, polje,...)  
VALUES (konstanta, konstanta,...)
```

ili:

```
INSERT  
INTO tabela (polje, polje,...)  
podupit;
```

```
3.59. INSERT  
INTO deo(ID_DETALJA, GRAD, TEŽINA)  
VALUES ('DEO7', 'Sombor', 2);
```

```
3.60. INSERT  
INTO deo  
VALUES ('DEO8', 'Zvezdica', 'Ružičasta', 14, 'Kula')  
Ovaj oblik je potencijalno opasan, zbog toga što je moguće promeniti strukturu tabele.
```

```
3.61. INSERT
```

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

```
INTO spj(ID_DOBAVLJAČA, ID_DETALJA, KOLIČINA)
VALUES ('D20', 'DEO20', 1000)
```

Ova naredba, takođe, može izazvati problema, jer sistem DB2 neće proveravati da li dobavljač S20 postoji u tabeli S i detalj P20 u tabeli P.

UNOS VIŠE ZAPISA

```
3.62. CREATE TABLE PRIVREMENA
      (ID_DETALJA CHAR(6),
       OBIM_ISPORUKE INTEGER);
INSERT
INTO PRIVREMENA (ID_DETALJA, OBIM_ISPORUKE)
      SELECT ID_DETALJA, SUM(KOLIČINA)
      FROM spj
      GROUP BY ID_DETALJA
```

Na kraju, moguće je izbrisati tabelu PRIVREMENA kada ona ne bude više potrebna.

```
DROP TABLE PRIVREMENA
```

VEŽBE

Kao i u prethodnim upitima, pretpostavlja se sledeća baza podataka:

$S = \{ \text{Dobavljač}(\{ID_DOBAVLJAČA, NAZIV, STATUS, GRAD\}, \{S_DOBAVLJAČA\}),$
 $Deo(\{ID_DETALJA, NAZIV, BOJA, TEŽINA, GRAD\}, \{ID_DETALJA\}),$
 $Proizvod(\{ID_PROIZVODA, NAZIV, GRAD\}, \{ID_PROIZVODA\}),$
 $SPJ(\{ID_DOBAVLJAČA, ID_DETALJA, ID_PROIZVODA, KOLIČINA\},$
 $\{ID_DOBAVLJAČA, ID_DETALJA, ID_PROIZVODA\}) \}$

3.63. Promenite boju svih crvenih delova u narandžastu.

```
UPDATE deo
SET BOJA = "narandžasta"
WHERE BOJA = "crvena"
```

3.64. Izbrišite sve proizvode, za koje nema isporuke detalja.

```
DELETE
FROM proizvod
WHERE ID_PROIZVODA NOT IN
      (SELECT ID_PROIZVODA
       FROM spj)
```

3.65. Povećajte obim isporuke za 10% za sve isporuke onih dobavljača, koji isporučuju bilo koji deo crvene boje.

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

```
CREATE TABLE CRVENI  
  (ID_DOBAVLJAČA CHAR(5));
```

```
INSERT INTO CRVENI(ID_DOBAVLJAČA)  
  SELECT DISTINCT ID_DOBAVLJAČA  
  FROM spj, deo  
  WHERE spj.ID_DETALJA = deo.ID_DETALJA  
  AND BOJA = "crvena"
```

```
UPDATE spj  
SET KOLIČINA = KOLIČINA * 1.1  
WHERE ID_DOBAVLJAČA IN  
  (SELECT ID_DOBAVLJAČA  
   FROM CRVENI)
```

```
DROP TABLE CRVENI
```

Primitimo da “rešenje” sa samo jednim upitom nije korektno. Zašto?

```
UPDATE spj  
SET KOLIČINA = KOLIČINA * 1.1  
WHERE ID_DOBAVLJAČA IN  
  (SELECT ID_DOBAVLJAČA  
   FROM spj, deo  
   WHERE spj.ID_DETALJA = deo.ID_DETALJA  
   AND BOJA = "crvena")
```

Odgovor: Ako fraza WHERE u naredbi UPDATE ili DELETE uključuje podupit, onda se u frazi FROM tog podupita ne sme navoditi tabela tog UPDATE ili DELETE. Analogno tome, u obliku naredbe INSERT sa podupitom u frazi FROM podupita ne sme se navoditi tabela, koja se pojavljuje u upitu INSERT. Tako, na primer, ako je potrebno obrisati sve dobavljače, čiji je status manji od prosečnog, onda sledeći upit nije dobar:

```
3.66. DELETE  
FROM dobavljač  
WHERE SASTAV <  
  (SELECT AVG(STATUS)  
   FROM dobavljač)
```

Umesto ovoga, treba:

```
SELECT AVG(STATUS)  
FROM dobavljač
```

SISTEMI ZA UPRAVLJANJE BAZAMA PODATAKA

Na primer, rezultat je 22.

Zatim:

```
DELETE  
FROM dobavljač  
WHERE SASTAV < 22
```

3.67. Izbrišite sve proizvode koji se proizvode u Kuli i sve odgovarajuće isporuke.

```
DELETE  
FROM spj  
WHERE "Kula" =  
    (SELECT GRAD  
     FROM proizvod p  
     WHERE p.ID_PROIZVODA = spj.ID_PROIZVODA)
```

```
DELETE  
FROM proizvod  
WHERE GRAD = "Kula"
```